

Xv6-2025 Report

GitHub仓库: <https://github.com/lpq1104/xv6-labs-2025.git>

本报告记录了 xv6 实验的环境配置、实现过程、测试结果与实验心得。正文包括工具与调试说明，以及 Utilities、System calls、Page tables、Traps、Copy on-write、Networking、Locks、File system 和 Mmap 九章实验。以下目录按正文顺序列出各章及实验项目，可点击跳转至对应内容。

目录

- [Tools & Guidance](#)
 - [Tools](#)
 - [Guidance](#)
- [Lab1 : Xv6 and Unix utilities](#)
 - [实验内容](#)
 - [Boot xv6](#)
 - [Sleep](#)
 - [sixfive](#)
 - [memdump](#)
 - [find](#)
 - [exec](#)
 - [Lab1实验得分](#)
- [Lab2 : System calls](#)
 - [Using gdb](#)
 - [Sandbox a command](#)
 - [Sandbox with allowed pathnames](#)
 - [Attack xv6](#)
 - [Lab2实验得分](#)
- [Lab3 : Page tables](#)
 - [Inspect a user-process page table](#)
 - [Speed up system calls](#)
 - [Print a page table](#)
 - [Use superpages](#)
 - [Lab3实验得分](#)
- [Lab4 : Traps](#)
 - [RISC-V assembly](#)
 - [Backtrace](#)
 - [Alarm](#)

- Lab4实验得分
 - Lab5 : Copy on-write
 - Implement copy-on write
 - Lab5实验得分
 - Lab6 : networking
 - Part One: NIC
 - Part Two: UDP Receive
 - Lab6实验得分
 - Lab7 : locks
 - Memory allocator
 - Read-write lock
 - Lab7实验得分
 - Lab8 : File system
 - Large files
 - Symbolic links
 - Lab8实验得分
 - Lab9 : Mmap
 - Mmap
 - Lab9实验得分
-

Tools & Guidance

Tools

安装WSL并启用虚拟化

1. 下载并安装适用于 Linux 的 Windows 子系统（Windows Subsystem for Linux）。然后从 Microsoft Store 添加 Ubuntu 20.04（Ubuntu 20.04 from the Microsoft Store）。
2. 检查WSL2要求：确认Windows版本是否符合要求。按下 Win+R 打开运行窗口，输入 "winver" 并检查 Windows 版本，确保版本号大于 1903。
3. 启用虚拟化命令：
以管理员的方式运行 `Powershell` 并在命令行中输入以下内容：

```
dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart
```

4. 下载X64的WSL2 Linux内核升级包并安装，将WSL的默认版本设置为WSL2：
以管理员的方式运行 `Powershell` 并在命令行中输入以下内容：

```
wsl --set-default-version 2
```

5. 安装Ubuntu:

在 Windows 中安装 Ubuntu 20.04 LTS 并设置用户账户。

- 安装命令：在命令提示符（CMD）中，以管理员的方式运行，输入以下命令来安装 Ubuntu 20.04 LTS：

```
wsl --install -d Ubuntu 20.04 LTS
```

- 根据系统提示，输入新 UNIX 用户名和密码。
- 安装完成后，系统显示安装成功的提示信息：

```
Installation successful!
```

软件源更新和环境准备

启动Ubuntu，安装本项目所需的所有软件，运行以下命令：

```
$ sudo apt-get update && sudo apt-get upgrade  
$ sudo apt-get install git build-essential gdb-multiarch qemu-system-misc gcc-riscv64-linux-gnu
```

测试安装

```
$ qemu-system-riscv64 --version  
$ riscv64-linux-gnu-gcc --version
```

编译内核

下载xv6内核源码

```
$ git clone git://github.com/mit-1 pdos/xv6-riscv.git
```

更新镜像源

```
$ sudo nano /etc/apt/sources.list  
$ sudo apt-get update
```

Guidance

调试技巧

检查进度

如果你的练习部分工作，请通过提交代码来检查你的进度。如果稍后出现问题，可以回滚到检查点并以较小的步骤前进。要了解更多关于Git的信息，请查看Git用户手册，或者，你可能会发现这个面向计算机科学的Git概述很有用。

测试失败

如果测试失败，请确保你了解为什么代码失败。插入打印语句直到你了解发生了什么。你可能会发现打印语句会生成大量你想搜索的输出；一种方法是在script中运行 `make qemu`（在你的机器上运行 `man script`），它将所有控制台输出记录到一个文件中，然后你可以搜索。不要忘记退出script。

使用GDB调试

在许多情况下，打印语句就足够了，但有时能够逐步执行一些汇编代码或检查堆栈上的变量是有帮助的。要使用gdb调试xv6，请在一个窗口中运行 `make qemu-gdb`，在另一个窗口中运行 `gdb`（或 `riscv64-linux-gnu-gdb`），设置断点，然后输入 `c`（继续），xv6将运行直到命中断点。（参见使用GNU调试器获取有用的GDB提示。）

如果你想查看编译器为内核生成的汇编代码或找到特定内核地址的指令，请参见 `kernel.asm` 文件，该文件在编译内核时由Makefile生成。（Makefile还为所有用户程序生成.asm文件。）

内核崩溃

如果内核崩溃，它将打印一条错误消息，列出崩溃时程序计数器的值；你可以搜索 `kernel.asm` 以查找程序计数器在崩溃时所在的函数，或者你可以运行 `addr2line -e kernel/kernel pc-value`（运行 `man addr2line` 获取详细信息）。如果你想获取回溯，请重新启动使用gdb：在一个窗口中运行 `make qemu-gdb`，在另一个窗口中运行 `gdb`（或 `riscv64-linux-gnu-gdb`），在panic中设置断点（`b panic`），然后输入 `c`（继续）。当内核命中断点时，输入 `bt` 获取回溯。

内核挂起

如果内核挂起（例如，由于死锁）或无法继续执行（例如，由于执行内核指令时的页面错误），你可以使用gdb找出挂起的地方。在一个窗口中运行 `make qemu-gdb`，在另一个窗口中运行 `gdb`（或 `riscv64-linux-gnu-gdb`），然后输入 `c`（继续）。当内核似乎挂起时，在qemu-gdb窗口中按 `Ctrl-C` 并输入 `bt` 获取回溯。

QEMU监视器

QEMU有一个“监视器”，可以让你查询仿真机器的状态。你可以通过输入 `control-a c`（“c”表示控制台）来访问它。一个特别有用的监视器命令是 `info mem`，用于打印页表。你可能需要使用 `cpu` 命令来选择 `info mem` 查看的核心，或者你可以使用 `make CPUS=1 qemu` 来启动qemu，以使其只有一个核心。

Lab1 : Xv6 and Unix utilities

本实验用于熟悉 xv6 及其系统调用。

实验内容

Boot xv6

1. 获取用于实验的 xv6 源代码并检出 util 分支:

```
$ git clone git://g.csail.mit.edu/xv6-labs-2021
Cloning into 'xv6-labs-2021'...
...
$ cd xv6-labs-2021
$ git checkout util
Branch 'util' set up to track remote branch 'util' from 'origin'.
Switched to a new branch 'util'
```

2. 构建并运行Xv6:

```
$ make qemu
riscv64-unknown-elf-gcc -c -o kernel/entry.o kernel/entry.S
riscv64-unknown-elf-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -DSOL_UTIL -MD -mcm
...

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$
```

如果在提示符下输入 `ls`，输出应类似于以下内容：

这些是 `mkfs` 在初始文件系统中包含的文件；大多数是可以运行的程序。刚才运行的其中一个程序是 `ls`。

要退出 `qemu`，请输入：`Ctrl-a x`。

Sleep

实验目的

1. 实现 UNIX 程序 `sleep` 以用于 xv6;

2. 实现该 `sleep` 程序应暂停指定的用户数量的时间片。时间片是由 xv6 内核定义的时间概念，即两个定时器芯片中断之间的时间。解决方案应在文件 `user/sleep.c` 中。

实验步骤

前期准备

在开始编码之前，阅读 [Xv6-book](#) 的第1章，并查看 `user/` 目录中的其他程序（例如 `user/echo.c`、`user/grep.c` 和 `user/rm.c`），以了解如何获取传递给程序的命令行参数。

编码步骤

1. 创建 `sleep.c` 文件

在 `user` 目录下，创建一个名为 `sleep.c` 的文件。

2. 编写 `sleep.c` 程序

在 `sleep.c` 中，编写一个程序，该程序接受一个命令行参数（表示 ticks 数量），并使当前进程暂停相应的 ticks 数量。在编写时需要注意以下几点：

- 如果用户没有提供参数或者提供了多个参数，程序应该打印出错误信息。
- 命令行参数作为字符串传递，可以使用 `atoi`（参见 `user/ulib.c`）将其转换为整数。
- 使用系统调用 `sleep`，最后确保 `main` 调用 `exit()` 以退出程序。

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
    if(argc != 2){
        fprintf(2, "usage: sleep ticks\n");
        exit(1);
    }

    int ticks = atoi(argv[1]);

    if(ticks < 0){
        fprintf(2, "sleep: ticks must not be negative\n");
        exit(1);
    }

    pause(ticks);
    exit(0);
}
```

3. 编辑 Makefile

将编写好的 `sleep` 程序添加到 Makefile 的 `UPROGS` 中。

- 打开 Makefile: `$ vim Makefile`
- 在 Makefile 中找到名为 `UPROGS` 的行, 这是一个定义用户程序的变量。在 `UPROGS` 行中, 添加 `sleep` 程序的目标名称: `$U/_sleep\`。

4. 编译并测试程序

使用 `make qemu` 编译 xv6 并启动虚拟机, 然后在 xv6 shell 中测试运行该程序:

- 在终端中, 运行 `make qemu` 命令编译 xv6。
- 在 xv6 shell 中运行编写的 `sleep` 程序。

5. 单元测试

使用 `./grade-lab-util sleep` 进行单元测试, 确保程序的正确性。

实验结果

从 xv6 shell 运行程序:

```
$ make qemu
...
init: starting sh
$ sleep 10
(一段时间内无事发生)
$
```

如果程序在以下情况下暂停, 则解决方案是正确的, 如图所示。

实验心得

1. 在本实验中, 成功实现了 xv6 操作系统下的 `sleep` 程序。实验的关键步骤包括正确解析命令行参数、调用 xv6 内核提供的 `sleep` 系统调用, 以及处理异常输入。
2. 通过编写 `sleep` 程序, 我学会了如何在 xv6 环境中创建用户级应用程序, 了解了时间片的概念以及系统调用的实现方式。这对于深入理解 xv6 内核及其调度机制有着重要意义。
3. 此外, 本次实验还锻炼了处理用户输入、进行错误检查和使用系统调用的能力, 这些技能在开发更复杂的操作系统应用时非常有用。未来可以进一步探索 xv6 中其他系统调用的实现, 增强对操作系统内核的全面理解。

sixfive

实验目的

1. 实现 xv6 用户级程序 `sixfive`, 依次读取命令行指定的文件, 输出其中所有能够被5或6整除的数字, 每个数字单独占一行。
2. 数字必须是由十进制数字组成、以指定分隔符或文件边界分隔的完整字符串。分隔符为 `" -\r\t\n./,"`, 因此 `/6,` 中的 `6` 应输出, 而 `xv6` 中的 `6` 不应输出。
3. 熟悉 `open`、`read` 和 `close` 系统调用, 掌握在 C 语言中逐字符读取文件、识别数字边界和处理异常的方法。实现代码位于 `user/sixfive.c`。

实验步骤

编码步骤

1. 创建 `sixfive.c` 文件

在 `user` 目录下创建 `sixfive.c`，引入类型定义、文件操作标志和用户态接口所需的头文件。

2. 编写 `sixfive.c` 程序

将程序分为分隔符判断、倍数判断、单文件处理和主函数四个部分。实际实现如下：

```

#include "kernel/types.h"
#include "kernel/stat.h"
#include "kernel/fcntl.h"
#include "user/user.h"

static int
is_separator(char c)
{
    char *separators = " -\r\t\n./,";

    return strchr(separators, c) != 0;
}

static void
print_if_multiple(int number)
{
    if(number % 5 == 0 || number % 6 == 0)
        printf("%d\n", number);
}

static int
process_file(char *path)
{
    int fd;
    int n;
    char c;
    int number = 0;
    int in_number = 0;
    int can_start = 1;

    fd = open(path, O_RDONLY);
    if(fd < 0){
        fprintf(2, "sixfive: cannot open %s\n", path);
        return -1;
    }

    while((n = read(fd, &c, 1)) > 0){
        if(c >= '0' && c <= '9'){
            if(in_number){
                number = number * 10 + (c - '0');
            } else if(can_start){
                number = c - '0';
                in_number = 1;
            }
        }

        can_start = 0;
    }

```

```

    } else if(is_separator(c)){
        if(in_number)
            print_if_multiple(number);

        number = 0;
        in_number = 0;
        can_start = 1;
    } else {
        number = 0;
        in_number = 0;
        can_start = 0;
    }
}

if(n < 0){
    fprintf(2, "sixfive: read error in %s\n", path);
    close(fd);
    return -1;
}

/*
 * 文件结尾被题目视为隐式分隔符。
 * 因此，文件最后如果正好是一个数字，需要在这里处理。
 */
if(in_number)
    print_if_multiple(number);

close(fd);
return 0;
}

int
main(int argc, char *argv[])
{
    int i;
    int failed = 0;

    if(argc < 2){
        fprintf(2, "usage: sixfive file...\n");
        exit(1);
    }

    for(i = 1; i < argc; i++){
        if(process_file(argv[i]) < 0)
            failed = 1;
    }
}

```

```
    exit(failed);  
}
```

程序中三个状态变量的作用如下：

- `number` 保存当前已读取的数值，每读入一个数字字符，通过 `number * 10 + (c - '0')` 更新。
- `in_number` 表示当前是否正在解析一个左边界合法的数字。遇到分隔符时，只有该变量为 1 才进行倍数判断，避免把连续分隔符之间的空内容作为数字 0 输出。
- `can_start` 表示当前位置是否允许开始一个新数字。初始值为 1，对应文件开头这一隐式分隔符；遇到指定分隔符后恢复为 1，遇到普通字母等其他字符后置为 0。
对于 `xv6`，读到字母后 `can_start` 为 0，因此后面的 `6` 不会开始一个数字；对于 `6abc`，虽然先读到了 `6`，但后续字母会清除当前状态，因此也不会输出。只有确认右边界是分隔符或正常的文件结尾后，才调用 `print_if_multiple`。

`read(fd, &c, 1)` 每次读取一个字节，返回 0 表示文件结束，返回负数表示读取错误。正常读到文件末尾时，程序额外处理尚未输出的数字，避免遗漏末尾没有换行符的情况。读取失败和正常结束两条路径都会调用 `close(fd)` 释放已打开的文件描述符。

3. 编辑 Makefile

将 `sixfive` 加入 Makefile 的 `UPROGS`：

```
$U/_sixfive\
```

4. 编译并测试程序

在实验目录中运行 `make qemu`，启动后在 `xv6 shell` 中执行 `sixfive sixfive.txt`。随后使用 `sixfive sixfive.txt README` 检查多个文件的处理，并检查不传入参数和文件不存在时的错误提示。

5. 单元测试

使用 `./grade-lab-util sixfive` 进行单元测试，确保程序的正确性。

实验结果

1. 示例文件测试

从 `xv6 shell` 运行程序，预期输出如下：

```
$ sixfive sixfive.txt  
5  
100  
18  
6  
$
```

实验截图：

2. 异常输入测试

不提供文件参数时，程序应输出用法提示；指定不存在的文件时，应输出打开失败信息。以下以不存在的 `missing.txt` 为例：

实验截图：

实验心得

1. 本实验的关键在于识别完整的数字及其边界。实现中通过 `in_number` 和 `can_start` 记录解析状态，使程序能够区分独立数字与字母数字混合字符串。指定分隔符出现时才完成当前数字的判断；遇到其他字符则丢弃当前候选数字，直到下一个合法分隔符后再允许开始解析。
2. 文件开头和结尾都需要单独考虑。初始化 `can_start = 1` 允许文件直接以数字开始；读取结束后检查 `in_number`，则保证末尾没有分隔符的数字也能被处理。倍数判断使用逻辑或运算，因此同时能被 5 和 6 整除的数字，例如 30，只输出一次。由于减号和小数点属于题目规定的分隔符，程序不会将它们解释为负数符号或小数点，例如 `-6` 中识别出的数值是 6。
3. 通过本实验，进一步熟悉了文件描述符、逐字符读取以及标准输出和标准错误的使用。当前实现不需要保存整个文件，额外空间开销为常数；但每个字符都调用一次 `read`，处理较大文件时可以考虑缓冲读取。另外，数值使用 `int` 累加，尚未检查过长数字导致的整数溢出，这是后续可以完善的地方。

memdump

实验目的

1. 进一步练习C语言指针的使用，理解指针与地址以及不同类型数据在内存中的表示方式。
2. 在 `user/memdump.c` 中实现 `memdump(char *fmt, char *data)` 函数，按照格式字符串 `fmt` 指定的方式输出 `data` 指向的内存内容。
3. 格式字符串中的每个字符决定如何输出后续的一部分数据，通过组合多个格式字符，可以依次输出包含多个字段的结构体。

实验步骤

编码步骤

1. 查看文件及实验要求

阅读 K&R 《C 程序设计语言（第2版）》第5.1至5.6节以及第6.4节，了解指针与地址、指针数组和指向结构体的指针。

查看已有的 `user/memdump.c`。程序不带参数时运行五组内置示例；带一个参数时，将其作为格式字符串，从标准输入读取最多512字节的数据，再调用 `memdump`。

2. 编写 `memdump` 函数

函数需要支持以下格式字符：

- `i`：将后续4字节解释为32位整数，以十进制输出。
- `p`：将后续8字节解释为64位整数，以十六进制输出。
- `h`：将后续2字节解释为16位整数，以十进制输出。

- **c**：将后续1字节解释为8位 ASCII 字符并输出。
- **s**：后续8字节保存一个字符串指针，输出该指针指向的字符串。
- **S**：当前位置直接保存以空字符结尾的字符串，输出该字符串。

使用循环遍历格式字符串，通过 `switch` 选择数据类型，完成输出后移动数据指针。
实际实现如下：

```
void
memdump(char *fmt, char *data)
{
    while(*fmt != '\0'){
        switch(*fmt){
            case 'i':
                printf("%d\n", *(int *)data);
                data += sizeof(int);
                break;

            case 'p':
                printf("%lx\n", *(uint64 *)data);
                data += sizeof(uint64);
                break;

            case 'h':
                printf("%d\n", *(short *)data);
                data += sizeof(short);
                break;

            case 'c':
                printf("%c\n", *(char *)data);
                data += sizeof(char);
                break;

            case 's':
                printf("%s\n", *(char **)data);
                data += sizeof(char *);
                break;

            case 'S':
                printf("%s\n", data);
                data += strlen(data) + 1;
                break;
        }

        fmt++;
    }
}
```

程序中指针操作的作用如下：

- `data` 的类型为 `char *`，可以按字节移动。读取整数时，先转换为对应类型的指针，再解引用取得数值，例如 `*(int *)data`。
- `s` 对应的位置保存的是字符串指针，因此通过 `*(char **)data` 取出字符串地址。输出后，原始数据指针移动8字节。
- `s` 对应的位置就是字符串内容，直接将 `data` 传给 `printf`。输出后通过 `strlen(data) + 1` 跳过字符串及末尾的空字符。

例如，`memdump("s", (char *)&s)` 传入的是指针变量的地址，需要先读取其中保存的字符串地址；而 `memdump("S", "a string")` 传入的就是字符串首地址，可以直接输出。每处理一个格式字符，`fmt++` 移动到下一项，直到遇到空字符结束。

3. 检查 Makefile

Makefile 已在 `util` 实验配置下将程序加入 `UPROGS`：

```
UPROGS += $U/_memdump
```

4. 编译并测试程序

在实验目录中运行 `make qemu`，启动后在 `xv6 shell` 中执行 `memdump`，检查内置示例。随后通过管道传入字符串，检查标准输入及不同格式的处理。

5. 单元测试

使用 `./grade-lab-util memdump` 进行单元测试，检查程序的正确性。

实验结果

1. 内置示例测试

从 `xv6 shell` 运行程序，预期输出如下。其中 Example 4 的第一行是十六进制地址，具体数值以实际运行结果为准：

```
$ memdump
Example 1:
61810
2025
Example 2:
a string
Example 3:
another
Example 4:
（实际十六进制地址）
1819438967
100
z
xyzyy
Example 5:
hello
w
o
r
l
d
$
```

实验截图：

2. 标准输入测试

通过管道传入字符串，按照 `hhcccc` 格式输出两个16位整数和四个字符，预期输出如下：

```
$ echo deadc0de | memdump hhcccc
25956
25697
c
0
d
e
```

实验截图：

实验心得

1. 本实验的关键在于根据格式字符正确解释内存，并在输出后移动到下一项数据。类型转换不会改变内存内容，而是决定解引用时读取多少字节、将其解释为什么类型。通过 `sizeof` 确定移动量，可以使读取范围与数据指针的移动保持一致。
2. 对于 `p`，当前实现读取 `uint64` 并移动8字节，但 `user/printf.c` 中的 `printint` 使用32位的 `uint x` 保存数值，会截断高32位。因此，虽然 `deadc0de` 对应的完整数值为

0x6564306364616564，当前输出仍为 64616564，与实验网页示例一致。这说明读取的数据宽度与最终显示宽度需要分别考虑。

3. 通过本实验，进一步理解了指针转换、解引用和内存布局之间的关系。当前函数按照格式连续读取数据，不会自动跳过结构体的填充字节，也没有数据长度检查，因此使用时需要保证格式与内存布局一致、数据范围足够且字符串指针有效。

find

实验目的

1. 编写一个简单版本的 UNIX 查找程序：查找目录树中带有特定名称的所有文件。相关解决方案应放在 `user/find.c` 文件中。
2. 理解文件系统中目录和文件的基本概念和组织结构。
3. 熟悉在 `xv6` 操作系统中使用系统调用和文件系统接口进行文件查找操作。
4. 应用递归算法实现在目录树中查找特定文件。

实验步骤

1. 首先查看 `user/ls.c` 以了解如何读取目录。
`user/ls.c` 中包含一个 `fmtname` 函数，用于格式化文件的名称。它通过查找路径中最后一个 `'/'` 后的第一个字符来获取文件的名称部分。如果名称的长度大于等于 `DIRSIZ`，则直接返回名称。否则，将名称拷贝到一个静态字符数组 `buf` 中，并用空格填充剩余的空间，保证输出的名称长度为 `DIRSIZ`。
2. 创建 `find.c` 文件：
在 `user` 目录下创建一个新的文件 `find.c`，然后编写如下代码：

```

#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"

char *fmtname(char *path) {
    static char buf[DIRSIZ + 1];
    char *p;

    // 找到最后一个斜杠之后的部分
    for (p = path + strlen(path); p >= path && *p != '/'; p--);
    p++;

    // 返回最后一个斜杠之后的部分
    if (strlen(p) >= DIRSIZ)
        return p;
    memmove(buf, p, strlen(p));
    memset(buf + strlen(p), 0, sizeof(buf) - strlen(p));
    return buf;
}

void find(char *path, char *target) {
    char buf[512], *p;
    int fd;
    struct dirent de;
    struct stat st;

    // 打开路径
    if ((fd = open(path, 0)) < 0) {
        fprintf(2, "find: cannot open %s\n", path);
        return;
    }

    // 获取文件状态
    if (fstat(fd, &st) < 0) {
        fprintf(2, "find: cannot stat %s\n", path);
        close(fd);
        return;
    }

    switch (st.type) {
    case T_FILE:
        if (strcmp(fmtname(path), target) == 0) {
            printf("%s\n", path);
        }
        break;
    }
}

```

```

case T_DIR:
    if (strlen(path) + 1 + DIRSIZ + 1 > sizeof(buf)) {
        printf("find: path too long\n");
        break;
    }
    strcpy(buf, path);
    p = buf + strlen(buf);
    *p++ = '/';
    while (read(fd, &de, sizeof(de)) == sizeof(de)) {
        if (de.inum == 0)
            continue;
        memmove(p, de.name, DIRSIZ);
        p[DIRSIZ] = 0;
        if (strcmp(de.name, ".") == 0 || strcmp(de.name, "..") == 0)
            continue;
        find(buf, target);
    }
    break;
}
close(fd);
}

int main(int argc, char *argv[]) {
    if (argc < 3) {
        fprintf(2, "Usage: find <path> <filename>\n");
        exit(1);
    }
    find(argv[1], argv[2]);
    exit(0);
}

```

- **fname函数：**

- 功能：从路径中提取文件名。它通过查找路径中最后一个斜杠后的部分来获取文件名。
- 处理逻辑：遍历路径字符串找到最后一个斜杠的位置，然后返回其后的部分。如果文件名长度小于DIRSIZ，则将其拷贝到静态缓冲区buf中，并用空格填充剩余的空间。

- **find函数：**

- 参数：路径（path）和目标文件名（target）。
- 功能：递归查找指定路径下的文件或目录，匹配目标文件名并打印其路径。
- 打开路径：尝试打开指定路径，如果失败则打印错误信息并返回。
- 获取文件状态：调用fstat获取文件的状态信息（类型、大小等）。
- 根据文件类型处理：
 - 如果是文件（T_FILE）：比较文件名，如果匹配则打印路径。

- 如果是目录（T_DIR）：遍历目录内容，跳过"."和"..", 并递归调用find函数查找子目录。
- main函数：
 - 参数检查：确保传入了正确数量的参数（路径和文件名）。
 - 调用find函数：传入用户输入的路径和文件名。
 - 退出程序：调用exit(0)退出程序。
- 3. 注意事项：
 - 递归处理：使用递归遍历目录和子目录，查找目标文件。
 - 跳过特殊目录：在遍历目录时，跳过"."和".."以避免无限递归。
 - 字符串处理：使用strcmp进行字符串比较，使用memmove和memset处理文件名。
 - 错误处理：在打开文件和获取文件状态时进行错误检查，确保程序的健壮性。
- 4. 更新 Makefile ：在 Makefile 中将 find 程序添加到 UPROGS 中。
- 5. 运行程序。

实验结果

输入以下命令：

```
$ echo > b
$ mkdir a
$ echo > a/b
$ find . b
$
```

运行截图：

程序输出符合预期，成功查找到目标文件并打印其路径。

实验心得

通过这次实验，我实现了一个简单的find命令，该命令能递归遍历目录树并查找目标文件。实验过程中，我学会了：

1. 目录读取：通过参考user/lis.c，我们了解了如何读取目录内容以及提取文件名。
2. 递归算法：递归方法让我们能够深入到子目录中进行查找，同时避免了无限递归的问题。
3. 文件系统接口的使用：在Xv6操作系统中，我们使用系统调用和文件系统接口来实现文件和目录的操作。
4. 字符串处理：C语言中的字符串处理需要特别注意，使用strcmp进行字符串比较，避免了直接使用==进行比较的错误。
5. 错误处理：在文件操作中，我们增加了错误检查，提高了程序的健壮性。

在实验中，我遇到了无限递归的问题。在遍历目录时，如果递归进入"."和".."目录，会导致无限递归，最终导致栈溢出。解决方案是在递归调用find函数之前，检查当前目录项是否为"."或"..", 如果是则跳

过。这可以通过在find函数中增加如下代码实现：

```
if (strcmp(de.name, ".") == 0 || strcmp(de.name, "..") == 0)
    continue;
```

在实验中，我还遇到了路径过长导致缓冲区溢出的问题。即在处理较长路径时，可能会出现缓冲区溢出的问题，导致程序崩溃或无法正确处理路径。只要在构建新的路径时，检查路径长度是否超过缓冲区大小，如果超出则打印错误信息并返回即可解决。修改后的代码如下：

```
if (strlen(path) + 1 + DIRSIZ + 1 > sizeof(buf)) {
    printf("find: path too long\n");
    break;
}
```

exec

实验目的

1. 熟悉 `fork`、`exec` 和 `wait` 系统调用，理解创建子进程、执行程序 and 等待子进程之间的关系。
2. 为 `find` 增加 `-exec cmd` 功能，对每个查找到的文件执行指定命令，并将文件路径作为命令的最后一个参数，而不是直接打印匹配到的文件名。
3. 在 `user/find.c` 中实现命令参数的构造与传递，使递归查找和命令执行能够结合使用。

实验步骤

编码步骤

1. 解析命令行参数

在原有 `find` 程序中增加 `-exec` 模式，命令格式为 `find path name -exec command args...`。其中 `argv[1]` 为查找路径，`argv[2]` 为目标文件名，`argv[3]` 为 `-exec`，从 `argv[4]` 开始是要执行的命令及其参数。

在 `main` 中区分基础模式和命令执行模式，并检查参数数量。
代码如下：


```

int
main(int argc, char *argv[])
{
    int cmd_argc;

    if(argc == 3){
        find(argv[1], argv[2], 0, 0);
        exit(0);
    }

    if(argc < 5 || strcmp(argv[3], "-exec") != 0){
        fprintf(2,
            "usage: find path name [-exec command args...]\n");
        exit(1);
    }

    cmd_argc = argc - 4;

    if(cmd_argc + 2 > MAXARG){
        fprintf(2, "find: too many exec arguments\n");
        exit(1);
    }

    find(argv[1], argv[2], &argv[4], cmd_argc);

    exit(0);
}

```

`MAXARG` 在 `kernel/param.h` 中定义，因此需要引入该头文件。除命令自身的参数外，还要为匹配文件的路径和末尾空指针预留两个位置，所以使用 `cmd_argc + 2 > MAXARG` 检查参数数组是否越界。

2. 编写命令执行函数

在 `find.c` 中添加 `run_command` 函数，先复制命令参数，再追加匹配到的文件路径，最后补上空指针。调用 `fork` 创建子进程，在子进程中调用 `exec`，父进程调用 `wait` 等待命令执行结束。代码如下：

```

static void
run_command(char *path, char **cmd_argv, int cmd_argc)
{
    char *exec_argv[MAXARG];
    int i;
    int pid;

    if(cmd_argc + 2 > MAXARG){
        fprintf(2, "find: too many exec arguments\n");
        return;
    }

    for(i = 0; i < cmd_argc; i++)
        exec_argv[i] = cmd_argv[i];

    exec_argv[cmd_argc] = path;
    exec_argv[cmd_argc + 1] = 0;

    pid = fork();

    if(pid < 0){
        fprintf(2, "find: fork failed\n");
        return;
    }

    if(pid == 0){
        exec(exec_argv[0], exec_argv);

        fprintf(2, "find: exec %s failed\n", exec_argv[0]);
        exit(1);
    }

    wait(0);
}

```

例如，运行 `find . wc -exec echo hi` 时，命令参数为 `{"echo", "hi"}`，匹配路径为 `./wc`，最终构造的参数数组为 `{"echo", "hi", "./wc", 0}`，相当于执行 `echo hi ./wc`。

`exec` 成功后会替换子进程的程序内容，不会返回原来的调用位置。因此，后面的错误提示和 `exit(1)` 只在执行失败时运行，避免子进程继续执行查找逻辑。

3. 修改查找函数

为 `find` 增加命令参数数组和参数数量，使每一层递归都能够获得执行命令所需的信息：

```

static void
find(char *path, char *target, char **cmd_argv, int cmd_argc)

```

找到匹配的普通文件时，根据 `cmd_argv` 判断当前模式：为空时打印路径，否则调用 `run_command`。

```
if(st.type == T_FILE){
    if(strcmp(base_name(path), target) == 0){
        if(cmd_argv == 0)
            printf("%s\n", path);
        else
            run_command(path, cmd_argv, cmd_argc);
    }

    close(fd);
    return;
}
```

遍历目录并拼接子路径后，将命令信息继续传递给递归调用：

```
find(buf, target, cmd_argv, cmd_argc);
```

4. 编译并测试程序

该功能直接在 `find.c` 中实现，使用已有的 `$U/_find` 编译目标。在实验目录中运行 `make qemu`，启动后执行 `find . wc -exec echo hi`，检查匹配路径是否正确追加到命令末尾。随后运行 `sh < findtest.sh`，检查递归查找和带参数命令的执行。

5. 单元测试

使用 `./grade-lab-util exec` 进行单元测试，检查程序的正确性。

实验结果

1. 命令执行测试

从 xv6 shell 运行程序，预期输出如下：

```
$ find . wc -exec echo hi
hi ./wc
$
```

实验截图：

2. 递归执行测试

输入以下命令：

```
$ sh < findtest.sh
```

`findtest.sh` 创建 `a/b`、`c/b` 和 `b` 三个内容为 `hello` 的文件，然后执行 `find . b -exec grep hello`。预期输出三行 `hello`，说明每个匹配文件都传递给了指定命令。输出中还会出现多个 `$`，这是因为 xv6 shell 在读取脚本时仍会打印提示符。

实验截图：

实验心得

1. 通过本实验，进一步理解了 `fork` 和 `exec` 的不同作用。`fork` 创建子进程，`exec` 则替换调用进程的程序内容。如果直接在执行查找的进程中调用 `exec`，第一次匹配后就无法继续遍历目录。因此，需要在子进程中执行命令，让父进程保留原有的查找流程。
2. 参数数组的构造是实现中的另一个关键。`exec_argv[0]` 保存程序名称，后面依次保存命令参数和匹配文件路径，最后必须以空指针结束。这里追加的是完整路径，例如 `./a/b`，这样命令才能访问子目录中的目标文件。通过在递归调用中传递 `cmd_argv` 和 `cmd_argc`，可以保证不同目录层级使用相同的命令配置。
3. 父进程在每次创建子进程后调用 `wait(0)`，等待当前命令完成并回收子进程，再继续查找下一个文件。这使多个匹配文件对应的命令按顺序执行，也避免了同时创建过多子进程。若 `exec` 失败，子进程打印错误信息并退出，父进程仍能继续处理后续匹配项。
4. 本次实现将目录遍历与命令执行分别放在 `find` 和 `run_command` 中，使基础查找和 `-exec` 模式共用递归逻辑。同时，参数数量检查和失败处理也让我认识到，调用系统接口时不仅要考虑正常执行流程，还需要明确出错后由哪个进程退出、哪个进程继续运行。当前父进程使用 `wait(0)`，没有收集子命令的退出状态，因此最终退出状态尚不能反映所有命令是否执行成功。

Lab1实验得分

Lab2 : System calls

本实验通过 GDB 调试、添加 `interpose` 系统调用和分析内存残留问题，了解用户程序进入内核的过程，以及操作系统如何限制进程行为和维护进程之间的隔离。

Using gdb

实验目的

1. 熟悉 GDB 的断点、单步执行、调用栈和寄存器查看功能，了解系统调用进入内核后的执行过程。
2. 结合异常寄存器和反汇编定位内核错误，完成 `answers-syscall.txt` 中的问题。

实验步骤

1. 启动调试环境

在实验目录中运行 `make qemu-gdb`，然后在另一个终端进入同一目录，启动 `gdb-multiarch`，加载生成的 `.gdbinit` 配置。若配置未自动加载，可执行 `source .gdbinit`。在 GDB 中输入：

```
(gdb) b syscall
(gdb) c
(gdb) layout src
(gdb) backtrace
```

在 `syscall` 入口设置断点，通过调用栈查看其调用者。使用 `n` 单步执行，越过 `struct proc *p = myproc();` 后查看进程和寄存器：

```
(gdb) p /x *p
(gdb) p /x p->trapframe->a7
(gdb) p /x $sstatus
```

2. 分析内核异常

按实验要求，临时将 `kernel/syscall.c` 中的 `num = p->trapframe->a7;` 替换为 `num = *(int *)0;`，重新编译运行，观察空地址访问引发的异常。根据输出中的 `sepc`，在 `kernel/kernel.asm` 中定位指令，并在 GDB 中通过 `b *地址` 设置断点，使用 `layout asm` 查看汇编。

在断点处查看 `p->name` 和 `p->pid`，确认发生异常时的进程。完成观察后恢复原语句；当前本地代码已使用正常的 `num = p->trapframe->a7;`。

实验结果

根据本地 `answers-syscall.txt`，记录的调试结果如下：

1. 调用 `syscall()` 的函数是 `usertrap()`。
2. `p->trapframe->a7` 为 `0xf`，即15，对应 `SYS_open`。第一个用户程序 `init` 调用 `open("console", O_RDWR)` 打开控制台。
3. `sstatus` 中的 `SPP` 位为0，表示陷入内核之前 CPU 处于用户模式。
4. 引发异常的指令为 `lw a3, 0(zero)`，尝试从虚拟地址0读取32位数据，目标寄存器 `a3` 对应变量 `num`。
5. 内核页表没有映射虚拟地址0，因此发生加载缺页异常。`scause=0xd` 对应异常编号13，`stval=0x0` 表明出错地址为0。
6. 异常发生时的进程为 `init`，进程号为1。

实验心得

通过本实验，我学会了将源码、汇编和寄存器状态结合起来分析问题。定位异常时，`sepc` 用于找到出错指令，`scause` 用于判断异常类型，`stval` 提供相关地址，调用栈则帮助确定执行路径。实际地址和寄存器分配可能随编译结果变化，因此应以本次构建为准，不能直接照搬网页中的地址。调试结束后也需要恢复人为引入的错误，避免影响后续实验。

Sandbox a command

实验目的

1. 添加 `interpose` 系统调用，通过掩码限制当前进程能够使用的系统调用，理解用户态接口与内核处理函数之间的连接方式。
2. 将限制保存在进程结构中，并在 `fork` 时传递给子进程。本节使用 `"-"` 作为路径参数，表示不设置路径例外。

实验步骤

编码步骤

1. 添加用户态接口和系统调用编号

将 `sandbox` 加入 Makefile 的 `UPROGS`：

```
$U/_sandbox\
```

在 `user/user.h` 中声明接口，在 `user/usys.pl` 中注册入口，并在 `kernel/syscall.h` 中分配编号，对应代码分别为：

```
int interpose(int, const char *);
```

```
entry("interpose");
```

```
#define SYS_interpose 22
```

编译时，`usys.pl` 生成汇编入口，将系统调用编号放入 `a7`，通过 `ecall` 进入内核。

2. 保存进程的沙箱状态

在 `kernel/proc.h` 的 `struct proc` 中添加以下字段。当前代码已包含下一节使用的路径字段：

```
uint64 syscall_mask;  
char allowed_path[MAXPATH];
```

在 `kernel/proc.c` 的 `allocproc` 和 `freeproc` 中初始化、清理状态，避免进程槽位复用保留旧限制：

```
p->syscall_mask = 0;  
p->allowed_path[0] = '\0';
```

在 `kfork` 中复制父进程的状态：

```
np->syscall_mask = p->syscall_mask;
safestrncpy(np->allowed_path, p->allowed_path,
            sizeof(np->allowed_path));
```

3. 实现并注册系统调用

在 `kernel/sysproc.c` 中实现 `sys_interpose`。通过 `argint` 读取掩码，通过 `argstr` 将用户传入的路径复制到内核缓冲区。当前完整实现如下：

```
uint64
sys_interpose(void)
{
    int mask;
    char path[MAXPATH];
    struct proc *p = myproc();

    argint(0, &mask);

    if(argstr(1, path, MAXPATH) < 0)
        return -1;

    p->syscall_mask = (uint)mask;
    safestrncpy(p->allowed_path, path, sizeof(p->allowed_path));

    return 0;
}
```

在 `kernel/syscall.c` 中声明处理函数，并在 `syscalls` 数组中增加映射项：

```
extern uint64 sys_interpose(void);
```

```
[SYS_interpose] sys_interpose,
```

4. 检查掩码并拒绝调用

在 `syscall()` 中取得调用编号，确认编号有效后，通过 `p->syscall_mask & (1ULL << num)` 判断对应位是否置1。命中限制且没有路径例外时，将 `p->trapframe->a0` 设为 `-1` 并返回，不执行处理函数。完整判断代码在下一节列出。

已有 `user/sandbox.c` 在子进程中先调用 `interpose`，再调用 `exec` 运行指定程序。限制保存在进程结构中，执行新程序后仍然有效；后续创建的子进程也会继承限制。

5. 编译并测试程序

使用 `make qemu` 编译运行，并通过 `./grade-lab-syscall sandbox_mask` 和 `./grade-lab-syscall sandbox_fork` 检查掩码及继承逻辑。

实验结果

输入以下命令，预期 `cat` 无法打开文件，而不需要打开文件的 `echo` 仍能输出：

```
$ sandbox 32768 - cat README
cat: cannot open README
$ sandbox 32768 - echo hello
hello
$
```

`SYS_open` 的编号为15，32768等于 `1 << 15`，因此这里只禁止 `open`。

实验截图：

实验心得

1. 本实验让我理解了添加系统调用需要同时连接用户态声明、汇编入口、调用编号和内核分发表。只实现处理函数并不足以让用户程序调用它，还需要逐项检查接口是否注册完整。
2. 沙箱状态的生命周期同样重要：新进程应从默认状态开始，子进程需要继承父进程的限制，释放进程时还要清理旧状态。将检查放在系统调用分发之前，可以统一拒绝受限操作，避免操作已经产生副作用后才返回错误。当前 `interpose` 会直接覆盖原有配置，如果它本身未被禁止，程序仍可再次调用它修改限制，因此这套教学实现并不等同于完整的安全沙箱。

Sandbox with allowed pathnames

实验目的

1. 扩展沙箱，为受到掩码限制的 `open` 和 `exec` 增加允许路径；当调用传入的路径与允许路径一致时，放行该调用。
2. 熟悉从用户空间获取字符串参数的方法，正确处理路径比较和特殊参数 `"-"`。

实验步骤

1. 保存允许路径

使用上一节的 `allowed_path[MAXPATH]` 字段保存路径。`sys_interpose` 通过 `argstr(1, path, MAXPATH)` 读取第二个参数，成功后再更新掩码和路径，避免参数读取失败时只更新了一部分状态。`kfork` 同时复制这两个字段。

2. 增加路径判断

在 `kernel/syscall.c` 中，只有命中掩码的 `open` 和 `exec` 才检查路径例外。两者的第一个参数都是路径，因此使用 `argstr(0, path, MAXPATH)` 读取。

当前 `syscall` 实现如下：


```

void
syscall(void)
{
    int num;
    struct proc *p = myproc();

    num = p->trapframe->a7;

    if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
        if(p->syscall_mask & (1ULL << num)) {
            int allow = 0;

            if(num == SYS_open || num == SYS_exec) {
                char path[MAXPATH];

                if(argstr(0, path, MAXPATH) >= 0 &&
                    strncmp(p->allowed_path, "-", MAXPATH) != 0 &&
                    strncmp(path, p->allowed_path, MAXPATH) == 0) {
                    allow = 1;
                }
            }

            if(!allow) {
                p->trapframe->a0 = -1;
                return;
            }
        }

        p->trapframe->a0 = syscalls[num]();
    } else {
        printf("%d %s: unknown sys call %d\n",
            p->pid, p->name, num);
        p->trapframe->a0 = -1;
    }
}

```

`allow` 初始为0，仅当路径读取成功、允许路径不是 "-" 且两个字符串完全相同时置1。未被掩码限制的调用直接执行；其他被限制的调用仍然返回 -1。

3. 编译并测试程序

使用 `make qemu` 运行程序，并通过 `./grade-lab-syscall sandbox` 检查沙箱相关测试，包括路径匹配、继承和特殊路径的处理。

实验结果

1. 允许路径测试

在 xv6 shell 中输入以下命令。先创建内容相同的文件 `x`，再检查沙箱是否只允许打开 `README`：

```
$ cat README > x
$ sandbox 32768 README grep xv6 README
$ sandbox 32768 README grep xv6 x
```

第一条沙箱命令应输出 `README` 中含有 `xv6` 的行，第二条应提示 `grep: cannot open x`。由于已提前创建 `x`，这里可以区分沙箱拒绝与文件不存在这两种情况。

实验截图：

实验心得

1. 路径参数来自用户空间，不能把用户地址直接当作内核字符串使用。通过 `argstr` 复制后再比较，可以检查字符串是否可访问以及长度是否符合限制。比较时也不能只检查首字符，否则会把 `---` 等合法文件名误认为特殊参数。
2. 当前实现比较的是路径字符串，而不是文件的实际身份，因此 `README` 与 `./README` 不会视为相同。这符合本实验的实现方式，但也说明路径限制并不是完整的文件权限模型。测试时除了验证允许路径能够访问，还应验证其他已存在文件被拒绝，以及限制在子进程中仍然生效。

Attack xv6

实验目的

1. 分析实验环境中未清理的物理页如何保留旧数据，理解内存初始化与进程隔离之间的关系。
2. 在 `user/attack.c` 中通过申请内存查找前一个 `secret` 进程留下的字符串，观察内核实现缺陷造成的信息泄露。

实验步骤

1. 分析数据残留的原因

当前实验通过 `LAB_SYSCALL` 条件编译跳过了 `kernel/vm.c` 中分配页面后的清零，以及 `kernel/kalloc.c` 中分配、释放页面时的填充操作。因此，页面重新分配给另一个进程时，部分旧内容仍可能存在。

`user/secret.c` 定义了8页大小的全局数组，并将参数写入偏移16字节的位置：

```
#define DATASIZE (8*4096)

char data[DATASIZE];
```

```
strcpy(data, "This may help.");
strcpy(data + 16, argv[1]);
```

`secret` 退出后，其页面被回收。`attack` 不直接访问其他进程的地址，而是在自己申请到的内存中寻找残留内容。

2. 编写 `attack.c`

当前实现调用 `sbrk` 申请8页内存，逐字节扫描，收集连续的字母和数字。遇到空字符且候选长度至少为2时输出一行，然后继续扫描。以下为本地实现，省略解释性注释：

```
#include "kernel/types.h"
#include "kernel/fcntl.h"
#include "user/user.h"
#include "kernel/riscv.h"
#define DATASIZE (8*4096)

int
main(int argc, char *argv[])
{
    char* buf = sbrk(DATASIZE);
    char ch[DATASIZE/4];
    int j = 0;

    for(int i = 0; i < DATASIZE; i++){
        char c = buf[i];
        if(c == '\0' && (j >= 2 && j < DATASIZE/4)){
            ch[j++] = '\0';
            printf("%s", ch);
            printf("\n");
            j = 0;
        }
        if( ((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') || (c >= '0' && c <= '9')) && j <
            ch[j++] = c;
        }
        else{
            j=0;
        }
    }
    exit(1);
}
```

`buf` 指向申请的区域，`ch` 保存候选字符串，`j` 记录候选长度。非字母数字字符会中断候选，空字符用于确认字符串结尾。代码没有固定秘密所在的页号，而是扫描整个申请区域，因此也可能输出其他符合条件的残留字符串。

3. 编译并测试程序

使用 `make qemu` 启动 xv6，先运行 `secret`，再运行 `attack`。评分脚本会运行两次 `attack`，检查输出中是否包含随机生成的秘密字符串。

实验结果

输入以下命令，观察输出中是否出现独占一行的 `xyzzzy`：

```
$ secret xyzzzy
$ attack
$ attack
```

当前实现会输出多个候选字符串，不能仅以“有输出”判断成功，应核对是否包含本次传给 `secret` 的参数。具体运行结果和评分结果以实验截图为准。

实验截图：

实验心得

1. 本实验说明，页表隔离并不能代替内存清理。即使两个进程没有同时共享同一页，物理页先后分配给不同进程时仍可能泄露旧数据。问题出现在页面重新交给用户程序之前没有清零，而不是用户程序能够随意访问其他进程的虚拟地址。
2. 对当前代码的检查也暴露出几个需要反思的地方：`sbrk` 的返回值没有检查；输出的是候选字符串，不能保证只输出秘密；最后固定使用 `exit(1)`，没有区分找到与未找到。更明显的是，`ch` 是8192字节的局部数组，而本地 `kernel/param.h` 中 `USERSTACK` 为1页，存在超出用户栈空间的风险。因此，仅凭代码意图不能认定实验已通过，需要结合实际运行检查；后续可以将候选缓冲区改为静态存储，并补充失败处理。
3. 这次实验让我认识到，功能测试没有报错并不代表隔离机制一定可靠。对于内存分配这类底层代码，还需要检查资源被重新使用时是否带入旧状态，以及测试是否覆盖了数据泄露这样的非功能性问题。

Lab2实验得分

Lab3 : Page tables

本实验通过观察用户页表、映射只读共享页、打印页表树和使用超级页，进一步理解 Sv39 地址转换及页面管理。

Inspect a user-process page table

实验目的

1. 观察用户进程的页表项，理解虚拟地址、物理地址和权限位之间的关系。
2. 区分代码页、数据页、栈、保护页及高地址处的特殊映射。

实验步骤

1. 在实验目录中运行 `make qemu`，进入 xv6 后运行 `pgtbltest`。
2. 查看 `print_pgtbl` 的输出。该函数通过已有的 `pgpte` 系统调用，打印进程地址空间最前面10页和最后面10页的页表项。
3. 结合 `kernel/riscv.h` 中的权限位、`kernel/memlayout.h` 中的地址布局以及 `answers-pgtbl.txt`，解释每项映射的用途。

实验结果

权限位中，`V=0x01` 表示有效，`R=0x02` 表示可读，`W=0x04` 表示可写，`X=0x08` 表示可执行，`U=0x10` 表示用户可访问；`A=0x40` 和 `D=0x80` 分别记录已访问和已修改状态。

实验心得

通过观察页表，我理解了“地址被映射”和“用户能够访问”并不是同一件事。保护页和 `trapframe` 都有有效映射，但缺少 `PTE_U`，因此用户不能直接访问。连续虚拟页也不要求对应连续物理页，页表负责记录二者之间的关系。比较两次输出时，应先检查映射用途和权限，而不是要求物理地址完全一致。

Speed up system calls

实验目的

某些操作系统（例如 Linux）通过在用户空间和内核之间共享一个只读区域的数据来加速某些系统调用。这消除了在执行这些系统调用时进入内核的需求。为了学习如何将映射插入页表，本实验在 `xv6` 中为 `getpid()` 系统调用实现此优化。

当每个进程创建时，在 `USYSCALL`（在 `memlayout.h` 中定义的一个虚拟地址）处映射一个只读页面。在该页面的开始位置存储一个 `struct usyscall`，并将其初始化为存储当前进程的 `PID`。用户空间侧已经提供了 `ugetpid()`，用于直接读取该映射。

实验步骤

编码步骤

1. 分配共享页

在 `kernel/proc.h` 的 `struct proc` 中保存页面地址：

```
struct usyscall *usyscall;
```

在 `allocproc` 分配 `trapframe` 后申请共享页，并在创建页表前完成初始化：

```

if((p->usyscall = (struct usyscall *)kalloc()) == 0){
    freeproc(p);
    release(&p->lock);
    return 0;
}

memset(p->usyscall, 0, PGSIZE);
p->usyscall->pid = p->pid;

```

整页清零可以防止向用户暴露旧数据。每个进程单独分配共享页，因此子进程使用自己的 PID，而不是直接共享父进程的 PID 页面。

2. 建立只读映射

在 `proc_pagetable` 中，将物理页映射到 `USYSCALL`，设置 `PTE_R | PTE_U`，不设置写入和执行权限。当前代码还在失败时撤销之前建立的特殊映射：

```

if(mappages(pagetable, USYSCALL, PGSIZE,
            (uint64)(p->usyscall), PTE_R | PTE_U) < 0){
    uvmunmap(pagetable, TRAMPOLINE, 1, 0);
    uvmunmap(pagetable, TRAPFRAME, 1, 0);
    uvmfree(pagetable, 0);
    return 0;
}

```

3. 释放页面与映射

在 `freeproc` 中释放共享页并清空指针：

```

if(p->usyscall)
    kfree((void*)p->usyscall);
p->usyscall = 0;

```

在 `proc_freepagetable` 中先撤销特殊映射，再释放普通用户内存和页表：

```

void
proc_freepagetable(pagetable_t pagetable, uint64 sz)
{
    uvmunmap(pagetable, TRAMPOLINE, 1, 0);
    uvmunmap(pagetable, TRAPFRAME, 1, 0);
    uvmunmap(pagetable, USYSCALL, 1, 0);
    uvmfree(pagetable, sz);
}

```

`uvmunmap` 的最后一个参数为0，表示这里只解除映射，不重复释放已经由 `freeproc` 管理的物理页。执行 `exec` 更换页表时，也可以重新映射同一个进程的共享页。

4. 编译并测试程序

使用 `make qemu` 启动 xv6，运行 `pgtbltest`，或使用 `./grade-lab-pgtbl ugetpid` 检查对应测试。

实验结果

`ugetpid_test` 比较 `getpid()` 与 `ugetpid()` 的返回值，并在64次创建子进程的过程中检查子进程PID。

输出为：

```
ugetpid_test starting
ugetpid_test: OK
```

实验心得

通过本实验，我理解了页面的分配、初始化、映射和释放需要作为一个完整过程考虑。用户侧只读并不意味着内核不能更新该物理页，内核仍可通过自己的映射写入数据。

Print a page table

实验目的

1. 实现 `vmprint(pagetable_t)`，以层次化格式输出有效页表项，帮助理解三级页表结构。
2. 输出各项对应的虚拟地址、PTE 和物理地址，并区分非叶子页表项与叶子映射。

实验步骤

编码步骤

1. 编写递归打印函数

在 `kernel/vm.c` 中逐项遍历页表，跳过未设置 `PTE_V` 的项。根据层级和索引计算虚拟地址，只对非叶子项递归。以下为本地实现，省略解释性注释：

```

static void
vmprintwalk(pagetable_t pagetable, int level, uint64 va)
{
    for(int i = 0; i < 512; i++){
        pte_t pte = pagetable[i];

        if((pte & PTE_V) == 0)
            continue;

        uint64 pteva =
            va | ((uint64)i << PXSHIFT(level));

        for(int depth = 0;
            depth < 3 - level;
            depth++){
            printf(" ..");

            printf("%p: pte %p pa %p\n",
                (void*)pteva,
                (void*)pte,
                (void*)PTE2PA(pte));

            if(level > 0 &&
                (pte & (PTE_R | PTE_W | PTE_X)) == 0){
                vmprintwalk(
                    (pagetable_t)PTE2PA(pte),
                    level - 1,
                    pteva
                );
            }
        }
    }
}

void
vmprint(pagetable_t pagetable)
{
    printf("page table %p\n", (void*)pagetable);
    vmprintwalk(pagetable, 2, 0);
}

```

PXSHIFT(level) 决定当前索引在虚拟地址中的位置。根层从 level 2 开始，每深入一层多输出一个 " .."。使用 %p 输出完整64位地址。

有效项中没有 R/W/X 位的项指向下一级页表；存在这些权限位的项是叶子映射，不能把其物理地址再当作页表继续遍历。这个判断也适用于下一实验中的 level 1 超级页。

2. 连接已有调用入口

`kernel/defs.h` 已声明 `vmprint`。2025版通过已有的 `kpgtbl()` 系统调用打印当前进程的页表，`kernel/sysproc.c` 中的调用为：

```
vmprint(p->pagetable);
```

`pgtbltest` 中的 `print_kpgtbl()` 调用该接口，因此不需要在 `exec.c` 中添加 `pid==1` 的打印条件。

3. 编译并测试程序

运行 `pgtbltest`，或通过 `./grade-lab-pgtbl print_kpgtbl` 检查输出格式。

实验结果

输出首行为页表地址，随后按层级显示有效项。

这些叶子映射与第一节观察的是同一类内容，权限解释也相同。新增的 `USYSCALL` 基本权限为 `0x13=V|R|U`，访问后可能为 `0x53`。非叶子项通常只有 `V` 位，保存的是下一级页表页地址。`vmprint` 展示完整的有效页表树，而 `print_pgtbl` 只列出选定虚拟页的查询结果。

实验截图：

实验心得

1. 页表遍历不能假设所有上层项都指向下一级页表。原报告的重重循环只检查有效位，加入超级页后可能把数据页误当作页表。递归时同时检查叶子权限位，才能正确决定是否继续访问。
2. 本实验还帮助我区分了页表索引和虚拟地址。实验要求输出虚拟地址，需要把各层索引组合起来，不能只打印当前循环下标。打印工具本身准确，才能用于排查后续映射和释放问题。

Use superpages

实验目的

1. 为满足2MB对齐且覆盖完整2MB的新增用户内存建立超级页映射，减少页表项数量和地址转换开销。
2. 支持超级页的分配、复制与回收，并在 `sbrk` 仅释放超级页尾部时，将保留区域降级为普通页。

实验步骤

编码步骤

1. 建立独立的超级页内存池
在 `kernel/kalloc.c` 中预留16个2MB物理块，共32MB，普通页和超级页分别使用 `kmem`、`supermem` 链表及锁管理：

```
#define NSUPERPAGES 16
#define SUPERPAGE_START \
    (PHYSTOP - NSUPERPAGES * SUPERPGSIZE)
```

kinit 分别初始化两个区域：

```
freerange(end, (void*)SUPERPAGE_START);
superfreerange((void*)SUPERPAGE_START, (void*)PHYSTOP);
```

superalloc 从超级页链表取出一个块； superfree 检查2MB对齐和地址范围后将块放回链表。实际分配函数如下：

```
void *
superalloc(void)
{
    struct run *r;

    acquire(&supermem.lock);
    r = supermem.freelist;
    if(r)
        supermem.freelist = r->next;
    release(&supermem.lock);

    if(r)
        memset((char*)r, 5, SUPERPGSIZE);

    return (void*)r;
}
```

源文件中的“20 MiB”注释与当前宏不一致，实际容量应按16乘2MB计算。两个分配器管理的物理范围分开，避免同一块内存重复分配。

2. 支持不同层级的叶子项

在 kernel/vm.c 中增加 walklevel，返回页表项时同时报告层级。若上层有效项已经设置 R/W/X，则直接返回，不再向下遍历：

```
if(*pte & (PTE_R | PTE_W | PTE_X)){
    if(levelout)
        *levelout = level;
    return pte;
}
```

walksuper 定位 level 1 的页表项， mapsuperpage 检查虚拟地址和物理地址都按2MB对齐，再建立 level 1 叶子映射：

```
*pte = PA2PTE(pa) | perm | PTE_R | PTE_V;
sfence_vma();
```

若该位置已有普通页表，代码先检查其512个项是否全部无效，只有空页表才允许回收并替换。`walkaddr` 也补充了超级页内部的4KB页偏移：

```
pa = PTE2PA(*pte);
if(level == 1)
    pa += PGROUNDDOWN(va) & (SUPERPGSIZE - 1);
return pa;
```

这样已有的 `copyin`、`copyout` 等按4KB处理数据时，也能定位到超级页中的正确位置。

3. 修改用户内存分配

在 `uvmalloc` 中检查当前位置是否对齐，以及剩余长度是否至少为2MB。满足条件时优先申请超级页：

```
if((a % SUPERPGSIZE) == 0 &&
    newsz - a >= SUPERPGSIZE){

    mem = superalloc();

    if(mem != 0){
        memset(mem, 0, SUPERPGSIZE);

        if(mapsuperpage(pagetable, a,
                        (uint64)mem,
                        PTE_R | PTE_U | xperm) == 0){
            a += SUPERPGSIZE;
            continue;
        }

        superfree(mem);
    }
}
```

不满足条件或超级页申请、映射失败时，继续使用普通4KB页。这样新增区间的未对齐前缀、尾部不足2MB的部分仍可正常分配。对用户可见的页面在映射前清零，避免泄露旧数据。

4. 修改 fork 复制逻辑

在 `uvmcopy` 中根据叶子层级选择复制大小。遇到 level 1 映射时，为子进程单独分配2MB块，复制内容并保留权限：

```

if(level == 1){
    if((i % SUPERPGSIZE) != 0)
        panic("uvmcopy: unaligned superpage");

    mem = superalloc();
    if(mem == 0)
        goto err;

    memmove(mem, (char*)pa, SUPERPGSIZE);

    if(mapsuperpage(new, i, (uint64)mem, flags) != 0){
        superfree(mem);
        goto err;
    }

    i += SUPERPGSIZE;
    continue;
}

```

普通页仍按4KB复制。失败时通过 `uvmunmap` 回收已经为子进程建立的映射，避免留下部分复制结果。

5. 处理完整释放和部分释放

`uvmunmap` 使用 `walklevel` 判断映射类型。范围覆盖完整超级页时调用 `superfree`，清除叶子项，并将遍历位置前进2MB。对于 `sbrk` 从超级页内部开始释放尾部的情况，调用 `demotesuperpage`。

本地降级实现会分配一个 level 0 页表，并为仍需保留的前缀逐页分配普通页、复制内容。关键代码为：

```
keep_pages = (va - superva) / PGSIZE;
```

```

memmove(mem,
        (void*)(superpa + i * PGSIZE),
        PGSIZE);

```

```
level0[i] = PA2PTE(mem) | flags;
```

```

*superpte = PA2PTE(level0) | PTE_V;
sfence_vma();
superfree((void*)superpa);

```

旧映射在准备完成前保持不变；分配失败时回收新申请的普通页和页表页。成功后保留数据位于普通页中，原2MB块整体归还超级页池。虽然函数前有“without copying”注释，但当前函数实际进

行了复制，不能按注释理解为原地拆分。

6. 编译并测试程序

运行 `pgtbltest`，重点检查 `superpg_fork` 和 `superpg_free`，或执行

`./grade-lab-pgtbl superpg`。再通过 `usertests -q` 检查普通页相关行为是否受到影响。

实验结果

`superpg_fork` 检查同一2MB范围内的查询是否返回相同 PTE，并检查父子进程中的映射和内容。`superpg_free` 检查释放最后4KB后，保留页的数据是否仍然存在、被释放页是否不再映射，以及后续逐页释放是否正确。

实验截图：

实验心得

1. 超级页的难点不只在于把分配单位改为2MB，而是所有依赖叶子层级和页面大小的操作都要同步处理。若查找函数继续把 level 1 叶子当作页表，或释放时仍按4KB反复处理同一叶子，就可能访问错误地址或重复释放。复制、地址转换和部分释放都需要检查。
2. 本地代码对空 level 0 页表的处理中，普通映射被移除后，页表页可能仍存在；这时不能简单地把所有有效上层项都当成重复映射，而应先检查下层是否为空，再决定是否替换。
3. 通过本实验，我认识到性能优化往往会增加资源管理复杂度。固定预留内存池便于保证连续性，但会限制普通页与超级页之间的灵活调配；复制式降级容易保持两个内存池独立，却需要额外内存和复制开销。后续完善时应同时考虑正确性、失败处理和空间成本。

Lab3实验得分

Lab4 : Traps

这个实验将探索系统调用如何通过陷阱（trap）来实现。

RISC-V assembly

实验目的

了解一些 RISC-V 汇编很重要。在 `xv6 repo` 中有一个文件 `user/call.c`。 `make fs.img` 会对其进行编译，并生成 `user/call.asm` 中程序的可读汇编版本。

实验步骤

1. 在xv6的命令行中输入运行 `make fs.img`，编译 `user/call.c` 程序，得到可读性比较强的 `user/call.asm` 文件。

2. 阅读 `call.asm` 中的 `g` , `f` , 和 `main` 函数。

回答下列问题：

Q1

Which registers contain arguments to functions?

哪些寄存器保存函数的参数？

在 RISC-V 架构中，寄存器 `a0` 到 `a7` 用于传递函数参数。具体地，前八个参数分别使用 `a0` 到 `a7` 这些寄存器传递。如果有更多参数，需要通过栈来传递。

Q2

Where is the call to function `f` in the assembly code for `main`? Where is the call to `g`? (Hint: the compiler may inline functions.)

`main` 的汇编代码中对函数 `f` 的调用在哪里？对 `g` 的调用在哪里（提示：编译器可能会将函数内联）

在 `main` 函数中没有直接的函数调用指令，而是内联了 `f` 和 `g` 的计算结果，函数 `f` 调用函数 `g` ，函数 `g` 使传入的参数加 3 后返回。

Q3

At what address is the function `printf` located?

`printf` 函数位于哪个地址？

查阅得到其地址在 `0x630` 。

Q4

What value is in the register `ra` just after the `jalr` to `printf` in `main`?

在 `main` 中 `printf` 的 `jalr` 之后的寄存器 `ra` 中有什么值？

`34: jalr 1536(ra) # 630 <printf>` 指令跳转到 `printf` 函数。

在执行 `jalr` 指令时，`ra` 寄存器会保存返回地址，也就是 `jalr` 指令的下一条指令的地址。在这种情况下：

`34: jalr 1536(ra)` 的下一条指令是 `38: li a0,0` 。

所以，在执行 `jalr` 指令后，`ra` 寄存器中保存的值是 `0x38` ，即 `main` 函数中 `printf` 调用之后的返回地址。

Q5

Run the following code.

```
unsigned int i = 0x00646c72;
printf("H%x Wo%s", 57616, &i);
```

What is the output? Here's an ASCII table that maps bytes to characters.

The output depends on that fact that the RISC-V is little-endian. If the RISC-V were instead big-endian what would you set `i` to in order to yield the same output? Would you need to change 57616 to a different value?

程序的输出是什么？这是将字节映射到字符的ASCII码表。

输出取决于RISC-V小端存储的事实。如果RISC-V是大端存储，为了得到相同的输出，你会把*i*设置成什么？是否需要将57616更改为其他值？

输出为 `HE110 World`。

若为大端对齐，`i` 需要设置为 `0x726c6400`，不需要改变 `57616` 的值（因为他是按照二进制数字读取的而非单个字符）。

Q6

In the following code, what is going to be printed after 'y='? (note: the answer is not a specific value.) Why does this happen?

在下面的代码中，“y=”之后将打印什么（注：答案不是一个特定的值）？为什么会发生这种情况？

```
printf("x=%d y=%d", 3);
```

在这段代码中，`printf` 函数的格式字符串要求两个整数参数，但实际只提供了一个整数参数 3。由于 `printf` 期望两个参数，而只提供了一个，这会导致未定义行为。具体来说，“y=”之后将打印什么取决于栈中紧接着的内容，这些内容可能是任何值。

这是由于以下几个原因导致的：

1. 参数不匹配：`printf` 函数的格式字符串包含两个 `%d`，但只提供了一个参数。这意味着函数会尝试从栈中获取第二个参数。
2. 未定义行为：C 语言标准中规定，当格式字符串的占位符数量与提供的参数数量不匹配时，行为是未定义的。这意味着编译器不会对这种情况做出任何保证，程序可能会打印垃圾值，崩溃，甚至可能正确运行（但这是偶然的）。
3. 栈内容未初始化：在调用 `printf` 时，函数会从栈中读取参数。由于没有提供第二个参数，`printf` 会读取一个未初始化的栈位置的值，导致打印出一个不可预测的值。

Backtrace

实验目的

实现一个回溯（`backtrace`）功能，用于在操作系统内核发生错误时，输出调用堆栈上的函数调用列表。这有助于调试和定位错误发生的位置。

实验步骤

1. 在文件 `kernel/riscv.h` 中添加内联函数 `r_fp()` 读取栈帧值。

2. 在 `kernel/printf.c` 文件中编写 `backtrace()` 函数以输出所有栈帧。函数的实现思路如下：
 - 通过调用 `r_fp()` 函数读取寄存器 `s0` 中的当前函数栈帧 `fp`。
 - 根据 RISC-V 的栈结构，`fp-8` 存放返回地址，`fp-16` 存放原栈帧。通过原栈帧可以得到上一级栈结构，依次类推，直到获取到最初的栈结构。
 - 需要考虑获取上一级栈帧的终止条件。RISC-V 的用户栈空间占一个页面，可以通过 `PGROUNDDOWN()` 和 `PGROUNDUP()` 计算得到一个地址所在页面的最高和最低地址。初始从寄存器 `s0` 读取到的栈帧 `fp` 是在用户栈空间中的地址，由此可以得到用户栈的页面最高和最低地址作为循环的终止条件。
3. 添加 `backtrace()` 函数原型到 `kernel/defs.h`。
4. 在 `kernel/sysproc.c` 的 `sys_sleep()` 函数中添加对 `backtrace()` 的调用。
5. 在 `kernel/printf.c` 的 `panic()` 函数中添加对 `backtrace()` 的调用。
6. 运行与测试。

实验结果

1. 在 `xv6` 中运行 `bttest`，输出 3 个栈帧的返回地址；退出 `xv6` 后运行 `addr2line -e kernel/kernel`，将 `bttest` 的输出作为输入，输出对应的调用栈函数，如下图所示。

根据输出的源码行号找对应的源码，发现就是 `backtrace()` 函数的所有调用栈的返回地址(函数调用完后的下一代码)。

实验心得

1. 对于栈帧指针的有效性检查，在遍历栈帧时，需要确保栈帧指针 `fp` 的有效性。通过检查 `fp` 是否在用户栈空间页面的范围内，确保访问的地址是合法的，避免出现异常访问和崩溃。
2. 在本次实验中，通过编写 `backtrace()` 函数并成功实现栈帧信息的输出，我深刻体会到了对底层栈结构的理解和对系统调用栈的掌握的重要性。特别是在解决获取上一级栈帧的终止条件和栈帧指针有效性检查的问题时，进一步加深了我对 RISC-V 架构和操作系统内部机制的认识。
3. 此外，在调试和验证过程中，通过使用 `addr2line` 工具将返回地址转化为源码行号，极大地方便了对调用栈信息的核对和分析。这不仅提高了代码的可靠性，还增强了我在系统编程和调试方面的能力，为今后处理类似问题积累了宝贵的经验。

Alarm

实验目的

本次实验将向 `xv6` 内核添加一个新的功能，即周期性地为进程设置定时提醒。这个功能类似于用户级的中断/异常处理程序，能够让进程在消耗一定的 CPU 时间后执行指定的函数，然后恢复执行。通过实现这个功能，我们可以为计算密集型进程限制 CPU 时间，或者为需要周期性执行某些操作的进程提供支持。

实验步骤

编码步骤

1. 添加系统调用接口

在 `user/user.h` 中声明注册提醒和恢复执行的接口：

```
int sigalarm(int ticks, void (*handler)());  
int sigreturn(void);
```

在 `user/usys.pl` 中添加汇编入口，并在 `kernel/syscall.h` 中定义编号，对应代码分别为：

```
entry("sigalarm");  
entry("sigreturn");
```

```
#define SYS_sigalarm 22  
#define SYS_sigreturn 23
```

在 `kernel/syscall.c` 中声明处理函数，并在 `syscalls` 数组中添加映射：

```
extern uint64 sys_sigalarm(void);  
extern uint64 sys_sigreturn(void);
```

```
[SYS_sigalarm] sys_sigalarm,  
[SYS_sigreturn] sys_sigreturn,
```

将测试程序加入 Makefile 的 `UPROGS`：

```
$U/_alarmtest\
```

2. 保存进程的提醒状态

在 `kernel/proc.h` 的 `struct proc` 中添加以下字段：

```
int alarm_interval;  
uint64 alarm_handler;  
int alarm_ticks;  
int alarm_in_handler;  
struct trapframe alarm_trapframe;
```

- `alarm_interval` 保存触发间隔，`alarm_handler` 保存用户处理函数地址。
- `alarm_ticks` 记录累计的时钟中断次数，达到间隔后触发提醒。
- `alarm_in_handler` 表示是否正在执行处理函数，用于防止重复进入。
- `alarm_trapframe` 保存被中断时的完整现场，供 `sigreturn` 恢复。

当前实现将备份现场直接放在进程结构中，不需要额外申请物理页。在 `kernel/proc.c` 的 `allocproc` 中初始化这些字段：

```
p->alarm_interval = 0;
p->alarm_handler = 0;
p->alarm_ticks = 0;
p->alarm_in_handler = 0;
memset(&p->alarm_trapframe, 0, sizeof(p->alarm_trapframe));
```

3. 实现 `sys_sigalarm`

在 `kernel/sysproc.c` 中读取间隔和处理函数地址，保存到当前进程，并重置计数：

```
uint64
sys_sigalarm(void)
{
    int ticks;
    uint64 handler;
    struct proc *p = myproc();

    argint(0, &ticks);
    argaddr(1, &handler);

    p->alarm_interval = ticks;
    p->alarm_handler = handler;
    p->alarm_ticks = 0;

    return 0;
}
```

`sigalarm(0, 0)` 将间隔设为0，从而关闭后续提醒。不能使用处理函数地址是否为0判断提醒是否启用，因为用户程序中的函数可能位于虚拟地址0。

此处不清除 `alarm_in_handler`，这样处理函数内部重新设置或关闭提醒时，也不会提前解除防重入状态。

4. 在时钟中断中触发处理函数

修改 `kernel/trap.c` 中 `usertrap` 的时钟中断分支。只有 `which_dev == 2` 时才更新计数，并且要求提醒已启用、当前不在处理函数中：

```

if(which_dev == 2){
    if(p->alarm_interval > 0 && p->alarm_in_handler == 0){
        p->alarm_ticks++;

        if(p->alarm_ticks >= p->alarm_interval){
            p->alarm_ticks = 0;
            p->alarm_in_handler = 1;

            memmove(&p->alarm_trapframe,
                    p->trapframe,
                    sizeof(struct trapframe));

            p->trapframe->epc = p->alarm_handler;
        }
    }

    yield();
}

```

触发时先保存原来的 `trapframe`，再将 `epc` 改为处理函数地址。随后通过正常的 `prepare_return` 和 `trampoline` 返回用户态，CPU 从处理函数开始执行，而不是在内核中直接调用用户函数。

`alarm_in_handler` 为1时不再累计和触发提醒，避免耗时较长的处理函数再次被自身打断。原有 `yield()` 保留，时钟中断仍然可以触发进程调度。

5. 实现 `sys_sigreturn`

用户处理函数完成任务后调用 `sigreturn()`，由内核恢复备份现场：

```

uint64
sys_sigreturn(void)
{
    struct proc *p = myproc();
    uint64 saved_a0 = p->alarm_trapframe.a0;

    memmove(p->trapframe,
            &p->alarm_trapframe,
            sizeof(struct trapframe));

    p->alarm_in_handler = 0;

    return saved_a0;
}

```

恢复完整 `trapframe` 可以同时恢复原来的程序计数器、栈指针和通用寄存器，使程序继续执行被中断的代码。清除防重入标志后，下一轮时钟中断重新开始计数。

这里必须返回保存的 `a0`，不能简单返回0。因为系统调用分发代码会把处理函数的返回值再次写入 `p->trapframe->a0`，返回0会覆盖刚刚恢复的寄存器内容。处理函数也不能只使用普通 `return` 结束，因为此次进入并不是常规函数调用，没有建立对应的调用返回现场。

6. 编译并测试程序

实验结果

1. 在 xv6 中执行 `alarmtest` .

实验截图：

实验心得

这次实验使我深入理解了操作系统的信号处理机制，通过实现定时提醒功能，我学会了如何在内核中添加系统调用、管理进程状态和处理中断，提升了系统编程和调试能力。

此外，这次实验也涉及到用户态和管理态的转换，我再次巩固了如何设置声明和入口使得二者连接。实验中遇到的挑战，如正确保存和恢复 `trapframe` 以及防止函数重入，使我认识到细致的状态管理和全面的测试对于系统开发的重要性。

通过测试程序，我明白在修改内核操作时，应确保不影响系统稳定性，即在实现定时中断处理功能时，要确保不会影响系统的正常运行，确保中断处理程序能够及时返回，避免影响其他中断和系统调度。进行充分的测试，我们才能确保定时中断处理不会导致系统崩溃或异常。这次实践不仅增强了我的理论知识，也提高了独立解决问题的能力。

Lab4实验得分

Lab5 : Copy on-write

虚拟内存提供了一种间接级别：内核可以通过将页表项（PTE）标记为无效或只读来拦截内存引用，导致页面错误，并且可以通过修改页表项来改变地址的含义。在计算机系统中，有一种说法是任何系统问题都可以通过增加一个间接层来解决。惰性分配实验提供了一个例子。本实验探讨了另一个例子：写时复制的fork。

- 主要问题：在xv6操作系统中，`fork()` 系统调用会将父进程的所有用户空间内存复制到子进程中。如果父进程占用的内存很大，复制过程可能会花费很长时间。更糟糕的是，这项工作通常大部分是浪费的。例如，在子进程中调用 `fork()` 后紧接着 `exec()`，会导致子进程丢弃复制的内存，可能大部分内存从未使用过。另一方面，如果父进程和子进程都使用某个页面，并且其中一个或两个都需要写入该页面，则确实需要进行内存复制。
- 解决方案：

为了解决上述问题，提出了写时复制（Copy-On-Write, COW）fork()机制。COW fork()的目标是推迟为子进程分配和复制物理内存页面，直到真正需要时才进行。

COW fork()仅为子进程创建一个页表，其中用户内存的页表项指向父进程的物理页面。COW fork()将父进程和子进程中的所有用户页表项标记为不可写。当任一进程尝试写入这些COW页面时，CPU会强制发生页面错误。内核页面错误处理程序检测到这种情况后，为发生错误的进程分配一个新的物理内存页面，将原始页面复制到新页面，并修改发生错误进程中的相关页表项，使其指向新页面，并将页表项标记为可写。当页面错误处理程序返回时，用户进程将能够写入其页面副本。

COW fork()使实现用户内存的物理页面释放变得更加复杂。一个给定的物理页面可能被多个进程的页表引用，只有在最后一个引用消失时才应被释放。

Implement copy-on write

实验目的

实验的主要目的是在 xv6 操作系统中实现写时复制（Copy-on-Write, COW）的 fork 功能。

传统的 fork() 系统调用会复制父进程的整个用户空间内存到子进程，而 COW fork() 则通过延迟分配和复制物理内存页面，只在需要时才进行复制，从而提高性能和节省资源。通过这个实验，你将了解如何使用写时复制技术优化进程的 fork 操作。

实验步骤

1. 修改 uvmcopy() 将父进程的物理页映射到子进程，而不是分配新页。原来 uvmcopy() 是将虚拟地址 [0, sz] 这个区间对应的物理内存的数据拷贝到新的物理内存中。现在不需要在这里申请新的物理内存，只需要将页表与父进程的物理内存进行映射就行，同时在子进程和父进程的 PTE 中清除 PTE_W 标志，设置 RSW 标志位。

```

int
uvmcopy(pagetable_t old, pagetable_t new, uint64 sz)
{
    pte_t *pte;
    uint64 pa, i;
    uint flags;
    // char *mem;

    for(i = 0; i < sz; i += PGSIZE){
        if((pte = walk(old, i, 0)) == 0)
            panic("uvmcopy: pte should exist");
        if((*pte & PTE_V) == 0)
            panic("uvmcopy: page not present");
        pa = PTE2PA(*pte);
        flags = PTE_FLAGS(*pte);
        // 清除PTE_W标志
        flags &= (~PTE_W);
        // 添加PTE_RSW标志
        flags |= PTE_RSW;
        // 清除父进程PTE的PTE_W标志
        *pte &= (~PTE_W);
        // 父进程PTE添加PTE_RSW标志
        *pte |= PTE_RSW;
        // 将父进程的物理内存映射到子进程的虚拟内存
        if(mappages(new, i, PGSIZE, (uint64)pa, flags) != 0){
            // kfree((void*)pa);
            goto err;
        }
        // 映射成功，父进程的物理内存引用计数增加
        mem_count_up(pa);
    }
    return 0;

err:
    uvmunmap(new, 0, i / PGSIZE, 1);
    return -1;
}

```

2. 在 riscv.h 中进行定义 PTE_RSW 标志位

```

#define PTE_X (1L << 3)
#define PTE_U (1L << 4) // 1 -> user can access
#define PTE_RSW (1L << 8) // 用这个标志位来表示cow的页面错误

```

3. 在 `kalloc.c` 文件中，参考 `kmem` 结构体对内存引用结构体进行了定义，同时定义了一些可能会用到的函数，例如增加引用计数、减少引用计数（若减少到0则函数返回真）、将引用计数设置为1，以及获取引用计数值。需要注意的是锁的使用，使用锁后需要及时释放锁。

`kalloc.c` 文件中：

```
// 内存引用计数的结构体
struct
{
    struct spinlock lock; // 若有多个进行同时对数组进行操作，需要上锁
    int mem_count[PHYSTOP/PGSIZE];
}mem_ref_struct;

int get_mem_count(uint64 pa){
    int count;
    acquire(&mem_ref_struct.lock);
    count = mem_ref_struct.mem_count[(uint64)pa / PGSIZE];
    release(&mem_ref_struct.lock);
    return count;
}

void mem_count_up(uint64 pa){
    acquire(&mem_ref_struct.lock);
    ++ mem_ref_struct.mem_count[(uint64)pa / PGSIZE];
    release(&mem_ref_struct.lock);
}

int mem_count_down(uint64 pa){
    int flag = 0;
    acquire(&mem_ref_struct.lock);
    if((-- mem_ref_struct.mem_count[(uint64)pa / PGSIZE]) == 0){
        flag = 1;
    }
    release(&mem_ref_struct.lock);
    return flag;
}

void mem_count_set_one(uint64 pa){
    acquire(&mem_ref_struct.lock);
    mem_ref_struct.mem_count[(uint64)pa / PGSIZE] = 1;
    release(&mem_ref_struct.lock);
}
```

4. 修改 `usertrap()` 以识别页面错误。在 `usertrap` 中，首先需要确定发生错误的虚拟地址是否来自写时复制（COW）。如果是，则需要根据内存引用计数来决定是否需要申请新的物理内存。如果引用计数不为1，可能有多个进程引用了同一段物理内存，此时需要申请新的物理内存，进行内存

拷贝并更新页表映射等操作。如果引用计数为1，则可能是父进程产生了页面错误，因为内存只剩一个引用，此时需要恢复物理内存的写权限，并清除RSW标志位。

需要注意以下几点：

- 在这个过程中，如果出现了任何失败，需要立即设置 `p->killed` 为 `1`，然后跳转到 `end` 处，退出并杀死进程。
- 在申请新的物理内存并进行映射之前，需要使用 `uvmunmap` 函数将虚拟地址与旧的物理内存进行解绑。

5. 内存引用计数相关的步骤在第一步已经做了相关的定义，接下来是一些使用的地方。

- 首先要对内存引用锁进行初始化，在 `kalloc.c` 文件中修改 `kinit()` 函数，初始化自旋锁。

```
void
kinit()
{
    initlock(&kmem.lock, "kmem");
    // 初始化mem_ref_struct的锁
    initlock(&mem_ref_struct.lock, "mem_ref");
    freerange(end, (void*)PHYSTOP);
}
```

在申请内存 `kalloc()` 函数，释放内存 `kfree()` 函数中，进行如下修改：

- `freerange` 函数中调用了 `kfree`，这个函数在系统内存初始化的时候调用，而且是在没有 `kalloc` 的前提下调用的，因为我们修改了 `kfree` 函数的逻辑，所以 `freerange` 函数中要先将内存引用计数置1。

```
void
freerange(void *pa_start, void *pa_end)
{
    char *p;
    p = (char*)PGROUNDUP((uint64)pa_start);
    for(; p + PGSIZE <= (char*)pa_end; p += PGSIZE){
        // 系统初始化时会将内存引用减1，所以这里先设为1
        mem_count_set_one((uint64)p);
        kfree(p);
    }
}
```

6. 下面对 `copyout` 函数进行修改。这里就是将内核物理内存copy到用户物理内存前需要检查一下用户物理内存（dst）是不是COW页面，如果是，则需要申请新的用户物理内存。这里只需要改动 `copyout` 而不需要改 `copyin` 是因为前者是内核拷贝到用户，是会对一个用户页产生写的操作，而后者是用户拷到内核，只是去读这个用户页的内容，COW页允许读。

7. 最后，一些函数需要在`defs.h`中进行声明。


```
int         get_mem_count(uint64 pa);
void        mem_count_up(uint64 pa);
int         mem_count_down(uint64 pa);
void        mem_count_set_one(uint64 pa);
pte_t*      cow_walk(pagetable_t , uint64 );
```

8. 编译并进行测试。

实验结果

实验心得

通过这个实验，我们深入了解了写时复制技术在操作系统中的实现原理及其优势。我们通过修改 xv6 操作系统，成功地实现了一个简单但有效的 COW fork 功能。虽然在实现过程中遇到了一些挑战，但最终的实现证明了写时复制在提高系统性能和节省资源方面的显著优势。通过进一步的优化和改进，可以使这一技术在实际应用中发挥更大的作用。

Lab5实验得分

Lab6 : networking

Part One: NIC

实验目的

1. 完成 `kernel/e1000.c` 中的 `e1000_transmit` 和 `e1000_recv`，实现网卡收发数据包。
2. 理解 DMA、描述符环和中断之间的关系，正确处理缓冲区的使用与释放。

实验步骤

编码步骤

1. 查看初始化代码

`e1000_init` 已配置发送环和接收环，两者均有16个描述符。网卡通过 DMA 读取发送缓冲区、写入接收缓冲区；驱动通过描述符的地址、长度和状态字段与硬件交换信息。

`TDT` 指向下一个待填写的发送位置，接收时从 `(RDT + 1) % RX_RING_SIZE` 开始检查。下标通过取模循环使用，不能只处理前16个包。

2. 实现发送函数

先检查描述符的 `DD` 位，确认硬件已经完成上一轮使用，再释放旧缓冲区并提交新包。以下为本地实现，省略解释性注释：

```

int
e1000_transmit(char *buf, int len)
{
    acquire(&e1000_lock);

    uint32 idx = regs[E1000_TDT];
    struct tx_desc *desc = &tx_ring[idx];

    if((desc->status & E1000_TXD_STAT_DD) == 0){
        release(&e1000_lock);
        return -1;
    }

    if(desc->addr != 0){
        kfree((void *)desc->addr);
    }

    desc->addr = (uint64)buf;
    desc->length = len;
    desc-> cso = 0;
    desc->cmd = E1000_TXD_CMD_EOP | E1000_TXD_CMD_RS;
    desc->status = 0;
    desc->css = 0;
    desc->special = 0;

    regs[E1000_TDT] = (idx + 1) % TX_RING_SIZE;

    release(&e1000_lock);
    return 0;
}

```

EOP 表示当前描述符是数据包的结束，RS 要求硬件回写完成状态。填写完成后推进 TDT，通知网卡处理新包。发送成功只表示已提交给网卡，缓冲区不能立即释放；当前实现等到该描述符被再次使用且 DD 置位时才回收旧页。

3. 实现接收函数

一次中断可能对应多个已完成描述符，因此循环处理直到遇到 DD 未置位的项。将旧页交给 net_rx 后，申请新页替换，再将描述符归还硬件：

```

static void
e1000_recv(void)
{
    while(1){
        uint32 idx = (regs[E1000_RDT] + 1) % RX_RING_SIZE;
        struct rx_desc *desc = &rx_ring[idx];

        if((desc->status & E1000_RXD_STAT_DD) == 0){
            break;
        }

        char *buf = (char *)desc->addr;
        int len = desc->length;

        net_rx(buf, len);

        char *newbuf = kalloc();
        if(newbuf == 0){
            panic("e1000_recv: kalloc");
        }

        desc->addr = (uint64)newbuf;
        desc->length = 0;
        desc->csum = 0;
        desc->status = 0;
        desc->errors = 0;
        desc->special = 0;

        regs[E1000_RDT] = idx;
    }
}

```

`net_rx` 可能释放旧页，也可能将其保存到 UDP 队列，因此驱动不能继续让网卡使用该页。已有 `e1000_intr` 确认中断后调用接收函数，协议栈则按以太网类型分发 ARP 和 IP 包。

4. 编译并测试程序

在一个 Ubuntu 终端运行 `python3 nettest.py txone`，另一个终端运行 `make qemu`，在 xv6 中执行 `nettest txone`。

接收测试需要先启动 xv6，再在 Ubuntu 终端执行 `python3 nettest.py rxone`。可以使用 `tcpdump -XXnr packets.pcap` 检查 ARP 请求、ARP 回复和 UDP 数据包。

实验结果

发送成功，宿主机测试脚本显示：

```
txone: OK
```

接收测试中，xv6 应先处理 ARP 请求并发送回复，再收到 IP 包，输出为：

```
arp_rx: received an ARP packet
ip_rx: received an IP packet
```

实验心得

本实验的关键是明确缓冲区什么时候交给硬件，什么时候可以回收。发送页需要等硬件完成读取，接收页交给协议栈后则需要替换，不能继续交给网卡写入。环形下标和循环接收也很重要，否则首次收发正常，并不代表持续通信正常。

锁的范围还要结合调用路径考虑。接收 ARP 后，协议栈会调用发送函数回复；如果接收时持有发送函数也要获取的同一把锁，就可能发生重复加锁。当前发送路径使用 `e1000_lock`，接收调用 `net_rx` 时未持有该锁。

当前实现仍有可完善之处：接收新页分配失败时直接 `panic`，没有采用丢包后继续运行的策略；发送环满时驱动返回 `-1`，但本地 `sys_send` 没有检查该返回值，可能报告成功并遗留未提交的缓冲区。后续应统一失败时的缓冲区释放责任。

Part Two: UDP Receive

实验目的

1. 完成 `kernel/net.c` 中的 `sys_bind`、`ip_rx` 和 `sys_recv`，使用户程序能够接收指定端口的 UDP 数据报。
2. 按端口保存到达的数据包，在队列为空时阻塞接收进程，并正确处理字节序、长度检查和内存释放。

实验步骤

编码步骤

1. 定义端口和接收队列
每个已绑定端口最多保存16个待接收数据包。本地实现使用如下结构，代码省略字段注释：

```

#define UDP_QUEUE_SIZE 16
#define MAX_UDP_PORTS NPROC

struct udp_packet {
    uint32 src_ip;
    uint16 src_port;
    char *payload;
    int payload_len;
    char *page;
};

struct udp_port {
    int used;
    uint16 port;
    int read_index;
    int write_index;
    int count;
    struct spinlock lock;
    struct udp_packet packets[UDP_QUEUE_SIZE];
};

static struct spinlock udp_table_lock;
static struct udp_port udp_ports[MAX_UDP_PORTS];

```

`payload` 指向包内的数据部分，`page` 保存原始分配地址，释放时必须使用后者。读写下标和 `count` 构成先进先出队列；端口表锁保护绑定信息，每个队列另有一把锁，避免一个端口的队列操作直接占用其他端口的队列锁。

2. 实现端口绑定

`netinit` 初始化端口表和各队列锁。`sys_bind` 获取端口参数，在端口表锁保护下先检查重复绑定，再寻找未使用槽位。初始化槽位时清零读写下标、计数和包记录，最后设置 `used=1`，确保槽位完全初始化后才可被查找到。

关键初始化代码为：

```

q->port = port;
q->read_index = 0;
q->write_index = 0;
q->count = 0;

```

若端口已经绑定或没有空闲槽位，返回 `-1`；成功返回0。`find_udp_port` 在表锁保护下按目标端口查找队列。可选的 `sys_unbind` 当前未实现资源回收，本实验必做测试不要求该功能。

3. 解析 IP 和 UDP 数据包

`ip_rx` 先检查最小长度，再根据 `ip_p` 判断是否为 UDP；UDP 包交给新增的 `udp_rx`，其他 IP 包直接释放。保留首次接收 IP 包的提示信息，评分脚本会检查该输出。

`udp_rx` 检查以太网、IP 和 UDP 头部是否完整，并根据 UDP 长度字段确认包未超出接收范围：

```

uint16 udp_len = ntohs(udp->ulen);

if(udp_len < sizeof(struct udp)){
    kfree(buf);
    return;
}

if((int)(sizeof(struct eth) + sizeof(struct ip) + udp_len) > len){
    kfree(buf);
    return;
}

uint16 destination_port = ntohs(udp->dport);
uint16 source_port = ntohs(udp->sport);
uint32 source_ip = ntohl(ip->ip_src);
int payload_len = udp_len - sizeof(struct udp);

```

网络头部使用网络字节序，而系统调用接口使用主机字节序，因此端口使用 `ntohs`，IP 地址使用 `ntohl`。负载长度根据 UDP 头确定，不能将以太网帧可能存在的填充当作用户数据。

4. 入队并唤醒进程

未绑定的端口直接丢包；队列已有16个包时丢弃新包，并释放其页面。允许入队时，将来源信息和数据位置保存在写下标处：

```

int index = q->write_index;
struct udp_packet *packet = &q->packets[index];

packet->src_ip = source_ip;
packet->src_port = source_port;
packet->payload = (char *)(udp + 1);
packet->payload_len = payload_len;
packet->page = buf;

q->write_index = (q->write_index + 1) % UDP_QUEUE_SIZE;
q->count++;

wakeup(q);

```

以上操作在 `q->lock` 保护下完成。队列容量按端口分别限制，某个端口积压不会直接占用另一个端口的16个队列位置。

5. 实现阻塞接收

`sys_recv` 获取目标端口、用户输出地址和 `maxlen`，拒绝负长度以及未绑定端口。获取队列锁后，队列为空则睡眠：

```

acquire(&q->lock);

while(q->count == 0){
    sleep(q, &q->lock);
}

```

`sleep` 配合队列锁完成条件检查与睡眠切换，包入队后通过 `wakeup(q)` 唤醒。醒来后继续用 `while` 检查，避免假设被唤醒就一定能取到包。

从读下标取出最早的包，复制长度为负载长度与 `maxlen` 的较小值。使用 `copyout` 分别写回源 IP、源端口和负载，例如负载复制代码为：

```

if(copy_len > 0 &&
    copyout(p->pagetable,
            user_buf,
            packet->payload,
            copy_len) < 0){
    failed = 1;
}

```

复制后保存原始页面地址，清空队列项，推进读下标并减少计数；解锁后释放页面。以下为清理流程中的关键语句：

```

char *page = packet->page;

```

```

q->read_index = (q->read_index + 1) % UDP_QUEUE_SIZE;
q->count--;

release(&q->lock);
kfree(page);

if(failed)
    return -1;

return copy_len;

```

每次接收移除一个完整数据报。用户缓冲区较小时只复制前缀，余下内容随该包一起丢弃；当前代码在复制失败时也会移除并释放该包。

6. 编译并测试程序

在 Ubuntu 的实验目录中先运行：

```

$ python3 nettest.py grade

```

另一个终端运行 `make qemu`，进入 xv6 后执行 `nettest grade`。

实验结果

综合测试检查基本收发、多个端口、连续通信、队列积压和内存回收，并通过已有 DNS 测试验证 UDP 接收链路。

实验截图：

实验心得

驱动收到数据包只是接收流程的开始，还需要完成协议解析、端口分发、队列保存以及向用户空间复制。通过本实验，我理解了中断上下文和进程上下文如何通过队列协作：中断侧入队后唤醒，进程侧没有数据时睡眠，双方用同一把队列锁保护状态，避免丢失唤醒。

内存管理需要覆盖所有出口。未绑定、队列满、格式不合法的包应立即释放；成功入队的包由接收系统调用释放。保存原始页面地址可以避免把包内的负载指针误传给 `kfree`。测试持续收发时，还需要观察内存是否逐渐减少，不能只验证某一次消息正确。

当前实现面向实验中的固定 IP 头格式，没有完整检查 IP 版本、可变头长、分片和校验和。接收等待循环也没有检查进程是否被终止，后续可以补充退出条件；可选 `unbind` 若要实现，还应同时设计等待进程的唤醒、队列清理和并发查找的生命周期管理。以上属于对当前实现的反思，不代表这些扩展已经完成。

Lab6实验得分

Lab7 : locks

在并发编程中，锁常用于解决同步与互斥问题，但在多核环境下，若不合理地使用锁，可能导致“锁竞争” (lock contention) 问题。为此，本实验旨在通过修改使用锁的数据结构来减少锁竞争的发生。

Memory allocator

实验目的

为了减少多核系统中的锁竞争并提升性能，可以重构内存分配器的设计。具体做法是为每个CPU分配一个独立的自由列表 (`free list`)，并且每个自由列表都有专属的锁。这样，不同CPU上的内存分配和释放操作可以并行执行，减少锁争用。同时，当某个CPU的自由列表耗尽时，它应能够从其他CPU的自由列表中获取空闲内存页。

实验步骤

1. 分析原有分配器的锁竞争问题

原来的 `kalloc()` 和 `kfree()` 共用一个空闲链表及其自旋锁，即使运行在不同CPU上，也需要等待同一把锁。`kalloc_test` 通过多个进程反复申请和释放内存，使这一问题更加明显。测试输出中的 `#acquire()` 表示获取锁的调用次数，`#test-and-set` 表示尝试获取锁时失败的循环次数。判断锁竞争是否降低，应关注后者及其总和，不能将单个CPU的获取次数减少直接理解为整体性能提高。

2. 修改 `kernel/kalloc.c` 中的数据结构及初始化函数

将原来的 `kmem` 改为大小为 `NCPU` 的数组，每个元素维护一把锁和一个空闲页链表。依次初始化后，仍调用 `freerange()` 将可用物理页交给分配器。

```
struct {
    struct spinlock lock;
    struct run *freelist;
} kmem[NCPU];

void
kinit()
{
    for(int i = 0; i < NCPU; i++){
        initlock(&kmem[i].lock, "kmem");
        kmem[i].freelist = 0;
    }
    freerange(end, (void*)PHYSTOP);
}
```

所有锁统一命名为 `"kmem"`，符合锁名以 `kmem` 开头的要求，无须额外增加 `lockname` 字段。`freerange()` 逐页调用 `kfree()`，因此启动阶段的空闲页首先进入执行初始化的CPU的链表，其他CPU通过后续的偷取操作获取页面。

3. 修改 `kfree()`，将页面归还到当前CPU的链表

保留原有的地址检查及页面填充操作，通过 `cpuid()` 找到当前CPU对应的链表，并在锁保护下将页面插入链表头部。

```

void
kfree(void *pa)
{
    struct run *r;

    if(((uint64)pa % PGSIZE) != 0 || (char*)pa < end || (uint64)pa >= PHYSTOP)
        panic("kfree");

    memset(pa, 1, PGSIZE);
    r = (struct run*)pa;

    push_off();
    int id = cpuid();

    acquire(&kmem[id].lock);
    r->next = kmem[id].freelist;
    kmem[id].freelist = r;
    release(&kmem[id].lock);

    pop_off();
}

```

`push_off()` 与 `pop_off()` 不仅需要包围 `cpuid()`，还要覆盖使用该CPU编号访问链表的过程，防止中途发生调度后继续使用原CPU编号。关闭本地中断并不能阻止其他CPU访问该链表，因此链表操作仍需加锁。

4. 修改 `kalloc()`，实现本地分配与跨CPU偷取

首先尝试从当前CPU的链表取出一页；若链表为空，则从下一个CPU开始循环检查其他链表，找到空闲页后立即返回。以下代码省略了源码中的注释：

```

void *
kalloc(void)
{
    struct run *r = 0;

    push_off();
    int id = cpuid();

    acquire(&kmem[id].lock);
    r = kmem[id].freelist;
    if(r != 0)
        kmem[id].freelist = r->next;
    release(&kmem[id].lock);

    if(r == 0){
        for(int offset = 1; offset < NCPU; offset++){
            int donor = (id + offset) % NCPU;

            acquire(&kmem[donor].lock);
            r = kmem[donor].freelist;
            if(r != 0)
                kmem[donor].freelist = r->next;
            release(&kmem[donor].lock);

            if(r != 0)
                break;
        }
    }

    pop_off();

    if(r != 0)
        memset((char*)r, 5, PGSIZE);

    return (void*)r;
}

```

本地实现每次只从其他CPU取出一页，并直接用于本次分配，没有额外的 `steal()` 函数，也没有将一半链表转移到本地。源码注释中提到了 `STEAL_BATCH`，但实际函数并未实现批量偷取，应以执行代码为准。

检查其他CPU之前已经释放本地锁，因此整个过程不会同时持有两把 `kmem` 锁，避免了相互偷取时因锁顺序不同而死锁。若本次遍历没有找到空闲页，则返回 `0`。取出的页面已从共享链表移除，填充操作放在释放锁之后，减少了临界区内的工作。

实验结果

在多核且负载较低的环境中启动 `xv6`，执行 `kallocetest`。检查 `test1` 至 `test4` 是否均输出 `OK`。
实验截图：

实验心得

1. 按CPU划分链表后，常见的本地分配和释放操作不再竞争同一把全局锁。但这些链表并不是完全私有的，其他CPU可能从中偷取页面，因此不能省略各自的锁。
2. 单页偷取的实现较简单，持有目标锁的时间短，但连续分配时可能频繁访问其他CPU。批量偷取可以减少访问次数，同时也增加链表拆分和转移的复杂度，需要根据实际竞争情况权衡，不能仅凭批量大小判断性能。

Read-write lock

实验目的

实现允许多个读者并发访问、写者独占访问的读写自旋锁。当有写者等待时，应阻止新的读者获取锁，避免读者不断进入而使写者长期无法执行。按照实验要求，在 `kernel/spinlock.h` 中定义状态，并在 `kernel/spinlock.c` 中实现初始化、读锁获取与释放、写锁获取与释放接口。

实验步骤

1. 定义读写锁结构并初始化

本地代码使用一把普通自旋锁 `guard` 保护读写锁的内部状态：`readers` 记录当前读者数，`writer_active` 表示是否有写者持锁，`waiting_writers` 记录等待写者数。以下定义位于 `LAB_LOCK` 条件编译范围内。

```
struct rwspinlock {
    struct spinlock guard;
    int readers;
    int writer_active;
    int waiting_writers;
};
```

在 `initrwlock()` 中初始化内部自旋锁，并将三个状态字段清零：

```

void
initrwlock(struct rwspinlock *rwlk)
{
    initlock(&rwlk->guard, "rwlock");
    rwlk->readers = 0;
    rwlk->writer_active = 0;
    rwlk->waiting_writers = 0;
}

```

2. 实现读锁的获取和释放

只有在没有活动写者、也没有等待写者时，才允许新的读者进入。条件检查与读者计数增加必须在同一个临界区中完成，否则写者可能在两者之间进入。

```

static void
read_acquire_inner(struct rwspinlock *rwlk)
{
    for(;;){
        acquire(&rwlk->guard);
        if(rwlk->writer_active == 0 &&
            rwlk->waiting_writers == 0){
            rwlk->readers++;
            release(&rwlk->guard);
            return;
        }
        release(&rwlk->guard);
    }
}

static void
read_release_inner(struct rwspinlock *rwlk)
{
    acquire(&rwlk->guard);
    if(rwlk->readers < 1)
        panic("read_release");
    rwlk->readers--;
    release(&rwlk->guard);
}

```

读者成功登记后立即释放 `guard`，因此不同CPU上的读者可以同时执行受读锁保护的代码。条件不满足时也要释放 `guard` 再重试，否则当前持锁者无法更新状态，等待将无法结束。

3. 实现写锁及写者优先策略

写者先增加 `waiting_writers`，使后续读者停止进入，再等待现有读者和写者退出。获得写锁时，在同一把 `guard` 下减少等待计数并设置活动标志。

```

static void
write_acquire_inner(struct rwspinlock *rwlk)
{
    acquire(&rwlk->guard);
    rwlk->waiting_writers++;
    release(&rwlk->guard);

    for(;;){
        acquire(&rwlk->guard);
        if(rwlk->readers == 0 &&
            rwlk->writer_active == 0){
            rwlk->waiting_writers--;
            rwlk->writer_active = 1;
            release(&rwlk->guard);
            return;
        }
        release(&rwlk->guard);
    }
}

static void
write_release_inner(struct rwspinlock *rwlk)
{
    acquire(&rwlk->guard);
    if(rwlk->writer_active == 0)
        panic("write_release");
    rwlk->writer_active = 0;
    release(&rwlk->guard);
}

```

等待计数只在进入循环前增加一次，不能在每次重试时重复增加。使用计数而非布尔值，可以在多个写者等待时准确保留等待状态：即使一个写者已经获得锁，其余写者仍会阻止新读者进入。

4. 在对外接口中配对关闭和恢复中断

```

void
read_acquire(struct rwspinlock *rwlk)
{
    push_off();
    read_acquire_inner(rwlk);
}

void
read_release(struct rwspinlock *rwlk)
{
    read_release_inner(rwlk);
    pop_off();
}

void
write_acquire(struct rwspinlock *rwlk)
{
    push_off();
    write_acquire_inner(rwlk);
}

void
write_release(struct rwspinlock *rwlk)
{
    write_release_inner(rwlk);
    pop_off();
}

```

外层的中断保护一直持续到读锁或写锁释放，不能在内部 `guard` 释放时就恢复中断。这样可以防止持锁期间被本CPU的中断路径打断后，再次等待同一读写锁造成死锁。普通自旋锁内部的中断控制通过嵌套计数与外层配合，调用者必须成对使用获取和释放接口。

实验结果

本地 `user/rwlktest.c` 创建4个子进程，内核测试的同步屏障也使用4个CPU，因此使用 `make CPUS=4 qemu` 启动，再执行 `rwlktest`。测试覆盖并发读、读写互斥、并发写、多把锁及写者优先等情况，全部通过时应输出：`rwlktest: 4/4 CPUs succeeded`

实验截图：

实验心得

1. `guard` 保护的是锁的状态，而不是整个读写临界区。读者完成计数更新后即可释放它，读写互斥由 `readers` 和 `writer_active` 表示。各状态的检查与修改都在 `guard` 下进行，利用原有自旋锁的原子操作及内存屏障保证同步，不能只依靠普通变量的读写。

2. 写者优先解决的是新读者持续进入造成的写者饥饿，并不保证写者按先来后到的顺序获得锁。如果写者持续到来，读者仍可能等待较长时间。此外，等待过程仍会反复获取 `guard`，因此该实现不是无锁算法，也不代表所有场景下都优于普通自旋锁。
3. 本次实验使我进一步理解了锁的粒度与共享数据组织方式之间的关系。内存分配器的优化不只是增加几把锁，而是将原来共享的链表按CPU拆分，使大部分操作访问不同的数据；同时仍需通过偷取机制处理空闲页分布不均的问题。

Lab7实验得分

Lab8 : File system

在本实验中，将为 xv6 文件系统添加对大文件和符号链接的支持。

Large files

实验目的

在不改变磁盘 `inode` 大小的前提下，增加二级间接块，使单个文件能够使用的数据块从268个扩展到65803个。通过修改块号映射与文件截断操作，理解文件逻辑块号到磁盘块号的转换，以及多级索引结构的分配和回收过程。

实验步骤

1. 修改块号相关宏定义

原来的 `inode` 包含12个直接块号和1个一级间接块号。为了给二级间接块保留位置，将 `kernel/fs.h` 中的 `NDIRECT` 改为11，并增加二级间接块容量的定义：

```
#define NDIRECT 11
#define NINDIRECT (BSIZE / sizeof(uint))
#define NDINDIRECT (NINDIRECT * NINDIRECT)
#define MAXFILE (NDIRECT + NINDIRECT + NDINDIRECT)
```

每个磁盘块为1024字节，一个 `uint` 块号占4字节，因此一个间接块可保存256个块号。扩展后文件最多包含 $11 + 256 + 256 * 256 = 65803$ 个数据块。这里的数量不包括用于保存块号的索引块。

2. 同步修改磁盘和内存中的 `inode`

将 `kernel/fs.h` 中 `struct dinode` 和 `kernel/file.h` 中 `struct inode` 的地址数组统一设置为：

```
uint addrs[NDIRECT+2];
```


数组仍有13个元素，磁盘 `inode` 的大小不变。其中 `addrs[0]` 至 `addrs[10]` 为直接块号，`addrs[11]` 为一级间接块号，`addrs[12]` 为二级间接块号。需要区分“第13个元素”和“下标13”，二级间接块对应的实际下标为 `NDIRECT + 1`。

本地实验分支的 `FSSIZE` 已设置为200000个块。修改布局后需重新生成 `fs.img`，避免继续使用按旧布局生成的文件系统镜像。

3. 在 `kernel/fs.c` 的 `bmap()` 中增加二级间接块映射

`bn` 是相对于文件起始位置的逻辑块号，而 `ip->addrs[]` 中保存的是磁盘块号。保留直接块和一级间接块的处理，在依次减去 `NDIRECT` 和 `NINDIRECT` 后，`bn` 就是二级间接区域内的偏移。

使用 `bn / NINDIRECT` 定位外层地址表中的条目，使用 `bn % NINDIRECT` 定位内层地址表中的数据块号。新增分支如下，省略源码注释：

```

if(bn < NDINDIRECT){
    uint first_index;
    uint second_index;

    first_index = bn / NINDIRECT;
    second_index = bn % NINDIRECT;

    if((addr = ip->addrs[NDIRECT + 1]) == 0){
        addr = balloc(ip->dev);
        if(addr == 0)
            return 0;
        ip->addrs[NDIRECT + 1] = addr;
    }

    bp = bread(ip->dev, addr);
    a = (uint*)bp->data;

    if((addr = a[first_index]) == 0){
        addr = balloc(ip->dev);
        if(addr == 0){
            brelse(bp);
            return 0;
        }

        a[first_index] = addr;
        log_write(bp);
    }

    brelse(bp);

    bp = bread(ip->dev, addr);
    a = (uint*)bp->data;

    if((addr = a[second_index]) == 0){
        addr = balloc(ip->dev);
        if(addr == 0){
            brelse(bp);
            return 0;
        }

        a[second_index] = addr;
        log_write(bp);
    }

    brelse(bp);

```

```
    return addr;
}
```

两级索引块和最终的数据块都按需分配，已有块号则直接复用。修改索引块后调用 `log_write()` 将变更纳入日志，每次 `bread()` 都要对应 `brelse()`，包括分配失败的返回路径。

例如，文件逻辑块号267是二级间接区域的第一个数据块，此时两级下标均为0；逻辑块号523对应该区域偏移256，两级下标分别为1和0。这样可以检查从一级间接区域进入二级间接区域，以及跨内层索引块时的边界是否正确。

4. 修改 `itrunc()`，释放二级间接块管理的全部空间

原有的直接块和一级间接块释放逻辑保持不变。新增部分先遍历外层索引，再遍历各内层索引块，依次释放数据块、内层索引块和最外层索引块：

```
if(ip->addrs[NDIRECT + 1]){
    bp = bread(ip->dev, ip->addrs[NDIRECT + 1]);
    a = (uint*)bp->data;

    for(i = 0; i < NINDIRECT; i++){
        if(a[i]){
            bp2 = bread(ip->dev, a[i]);
            a2 = (uint*)bp2->data;

            for(j = 0; j < NINDIRECT; j++){
                if(a2[j])
                    bfree(ip->dev, a2[j]);
            }

            brelse(bp2);
            bfree(ip->dev, a[i]);
        }
    }

    brelse(bp);
    bfree(ip->dev, ip->addrs[NDIRECT + 1]);
    ip->addrs[NDIRECT + 1] = 0;
}

ip->size = 0;
iupdate(ip);
```

`bp2` 和 `a2` 分别用于内层索引块的缓冲区及地址表，避免覆盖外层遍历所需的 `bp` 和 `a`。仅释放数据块而遗漏索引块会造成磁盘空间泄漏；最后清空入口和文件大小，并通过 `iupdate()` 更新磁盘上的 `inode`。

实验结果

重新编译并启动 `xv6`，执行 `bigfile`。本地评分脚本要求写入65803个块并完成读回检查，通过输出为：

```
wrote 65803 blocks
reading bigfile
bigfile done; ok
```

实验截图：

实验心得

- 通过此次实验，我理解了如何在有限的 `inode` 空间内管理更多数据块。二级间接块并不是扩大地址数组，而是用一个直接块号的位置换取更深一层的索引。实现时应先明确逻辑块号的范围，再计算各级下标，避免将文件内的块号与磁盘块号混淆。
- 文件大小扩展后，回收逻辑也必须同步修改。`bmap()` 负责建立索引关系，`itrunc()` 则需要沿相同的结构逐层回收。检查代码时不能只关注能否写入大文件，还要关注索引块是否释放、缓冲区是否归还，以及失败路径是否遗漏资源。

Symbolic links

实验目的

实现 `symlink(target, path)` 系统调用，在 `path` 创建保存目标路径的符号链接，并使 `open()` 能够跟随链接打开目标文件。支持 `O_NOFOLLOW` 打开链接本身，通过深度限制处理循环链接。本实验只要求 `open()` 跟随符号链接，不要求实现指向目录的符号链接访问。

实验步骤

- 添加系统调用接口及相关常量

在 `kernel/syscall.h` 中分配系统调用号：

```
#define SYS_symlink 22
```

在 `kernel/syscall.c` 中增加函数声明及系统调用表项：

```
extern uint64 sys_symlink(void);
```

```
[SYS_symlink] sys_symlink,
```

在 `user/user.h` 中增加用户接口，在 `user/usys.pl` 中生成调用桩：

```
int symlink(const char*, const char*);
```

```
entry("symlink");
```

分别在 `kernel/stat.h` 和 `kernel/fcntl.h` 中增加文件类型与打开标志：

```
#define T_SYMLINK 4
```

```
#define O_NOFOLLOW 0x800
```

`O_NOFOLLOW` 使用独立的标志位，能够与其他打开方式按位或组合。将 `$U/_symlinktest` 加入 `Makefile` 的用户程序列表，以便在 `xv6` 中运行测试。

2. 在 `kernel/sysfile.c` 中实现 `sys_symlink()`

符号链接拥有自己的 `inode`，其数据块保存目标路径。创建时不查找目标，因此目标尚不存在也可以创建链接。具体代码如下：

```

uint64
sys_symlink(void)
{
    char target[MAXPATH];
    char path[MAXPATH];
    struct inode *ip;
    int len;

    if(argstr(0, target, MAXPATH) < 0 ||
        argstr(1, path, MAXPATH) < 0)
        return -1;

    begin_op();

    ip = create(path, T_SYMLINK, 0, 0);
    if(ip == 0){
        end_op();
        return -1;
    }

    len = strlen(target) + 1;

    if(writei(ip, 0, (uint64)target, 0, len) != len){
        iunlockput(ip);
        end_op();
        return -1;
    }

    iunlockput(ip);
    end_op();
    return 0;
}

```

`create()` 返回的 `inode` 已加锁，可直接交给 `writei()`。写入长度包含字符串结尾的 `'\0'`，参数中的 `0` 表示源地址位于内核空间。正常完成返回0，参数读取、创建或写入失败返回-1。

`begin_op()` 和 `end_op()` 将文件系统修改纳入日志事务；`iunlockput()` 释放锁并减少内存引用计数，并不等于无条件删除磁盘文件。

3. 在 `sys_open()` 中跟随符号链接

本地代码在未设置 `O_CREATE` 的分支中，通过 `namei()` 查找路径，再调用 `ilock()` 获取锁。`namei()` 本身只返回带引用的 `inode`，不能将其误认为已经加锁。

若未设置 `O_NOFOLLOW`，则循环读取链接内容并查找下一个目标，直到遇到非符号链接文件。新增部分如下，省略源码注释：

```

if((omode & O_NOFOLLOW) == 0){
    depth = 0;

    while(ip->type == T_SYMLINK){
        if(depth++ >= 10){
            iunlockput(ip);
            end_op();
            return -1;
        }

        n = readi(ip, 0, (uint64)path, 0, MAXPATH);
        if(n <= 0){
            iunlockput(ip);
            end_op();
            return -1;
        }

        if(n < MAXPATH)
            path[n] = '\0';
        else
            path[MAXPATH - 1] = '\0';

        iunlockput(ip);

        if((ip = namei(path)) == 0){
            end_op();
            return -1;
        }

        ilock(ip);
    }
}

```

`depth` 最多允许跟随10个符号链接，超过限制就返回错误。这种方式不精确区分环与过长的合法链接链，但可以避免无限跟随，符合实验要求；本地实现没有额外的 `NSYMLINK`、访问记录数组或哈希表。

每轮先读取目标路径，再释放当前 `inode` 的锁和引用，随后查找并锁定目标。不能持有当前锁直接查找并锁定下一个目标，否则自引用链接可能造成重复获取同一把锁而死锁。目标不存在时，`namei()` 返回空指针，`open()` 返回-1。

4. 保留不跟随链接时的行为及后续打开流程

设置 `O_NOFOLLOW` 后跳过上述循环，后续文件对象直接引用符号链接的 `inode`。成功打开时，`f->ip = ip` 保存引用，函数只调用 `iunlock(ip)`，由文件关闭流程负责后续引用释放。对于截断操作，本地代码只允许截断普通文件：

```
if((omode & O_TRUNC) && ip->type == T_FILE)
    itrunc(ip);
```

因此，跟随链接打开普通文件时截断的是目标文件，使用 `O_NOFOLLOW` 时不会因 `O_TRUNC` 清空链接路径。`link()` 和 `unlink()` 不增加跟随逻辑，仍操作符号链接本身，删除符号链接不会直接删除目标文件。

实验结果

在 `xv6` 中执行 `symlinktest`，检查普通符号链接与并发创建测试。通过时输出：

```
Start: test symlinks
test symlinks: ok
Start: test concurrent symlinks
test concurrent symlinks: ok
```

实验截图：

实验心得

1. 符号链接与硬链接的区别不仅在于表现形式。硬链接增加同一个 `inode` 的目录引用，而符号链接保存目标路径，打开时再查找该路径。因此目标不存在不影响符号链接创建，但会影响后续打开；这也说明创建链接与验证目标应当分开处理。
2. 这一部分需要同时关注锁和引用计数。`ilock()`、`iunlock()` 控制对 `inode` 内容的访问，`iput()` 则减少内存引用。跟随链接时要交还旧引用，成功打开后又要保留最终引用，不能简单地在所有返回位置使用同一种释放方式。
3. 深度限制实现简单，也能够避免循环链接导致无限等待，但会拒绝超过限制的合法链接链。当前代码仅在 `open()` 的非 `O_CREATE` 分支中跟随链接，不能据此认为已经实现完整的符号链接语义。实验报告需要明确实现范围，而不是将实验中的简化处理描述为通用方案。
4. 检查失败路径时还发现一个值得改进的地方：`create()` 已建立目录项后，如果 `writei()` 写入目标路径失败，当前 `sys_symlink()` 虽然返回-1，却没有撤销新建的目录项。`end_op()` 也不是事务回滚操作，因此可能留下不完整的链接。后续可以补充磁盘空间不足等异常测试，并完善创建失败时的清理，不能仅凭正常功能测试判断所有错误路径都已处理完整。

Lab8实验得分

Lab9 : Mmap

Mmap

实验目的

本次实验的目标是为 xv6 操作系统添加 `mmap` 和 `munmap` 系统调用，以实现进程地址空间的精细控制。通过这两个系统调用，可以实现内存映射文件的功能，包括共享内存和将文件映射到进程的地址空间等。这对于理解虚拟内存管理和页面错误处理机制具有重要意义。

实验步骤

1. 增加 `mmap` 和 `munmap` 的 `system call` 声明。

```
...
#define SYS_close  21
#define SYS_mmap   22
#define SYS_munmap 23
```

```
...
extern uint64 sys_uptime(void);
extern uint64 sys_mmap(void);
extern uint64 sys_munmap(void);
```

```
...
[SYS_close]   sys_close,
[SYS_mmap]    sys_mmap,
[SYS_munmap]  sys_munmap,
```

```
...
char* mmap(char *addr, int length, int prot, int flags, int fd, int offset);
int munmap(char *addr, int length);
```

```
...
entry("uptime");
entry("mmap");
entry("munmap");
```

2. `Makefile` 中增加对 `mmaptest` 的编译。
3. 使用 `VMA` 存储 `mmap` 映射信息，在 `struct proc` 结构体中增加 `vma_pool` 字段。

```
#define MAX_VMA_POOL 16
```

```
struct VMA{  
    int used;  
    uint64 addr;  
    uint32 length;  
    int prot;  
    int flags;  
    int offset;  
    struct file *f;  
};
```

4. 在进程初始化函数（`proc.c` 中的 `allocproc` 函数）中增加对 `vma_pool` 的初始化。
5. 实现在 `vma_pool` 中分配和释放 `VMA` 的逻辑（`proc.c`），所谓的“分配”其实也就是在 `vma` 数组中寻找一个未使用的位置。将以上函数声明写到 `defs.h` 中。
6. 实现 `sys_mmap()`。
 - i. 解析传入的参数：首先，`sys_mmap()` 函数需要解析调用时传入的参数。这些参数通常包括文件描述符、映射的内存大小、保护标志（如可读、可写）、映射标志（如共享、私有）、偏移量等。这些参数将决定内存映射的行为和特性。
 - ii. 权限检查：接下来，函数需要检查当前进程是否有权限进行 `mmap` 操作。这包括检查映射的权限（如读、写、执行）是否与文件的权限一致，并确认进程是否有足够的资源进行内存映射。
 - iii. 分配 `VMA`（虚拟内存区域）：从当前进程的 `VMA` 池中分配一个空的 `VMA`（Virtual Memory Area）条目，并将解析得到的 `mmap` 参数信息填充到这个 `VMA` 中，包括映射的地址、大小、权限、文件偏移量等。`VMA` 是用于描述进程地址空间中一段连续虚拟内存的结构。
 - iv. 分配内存：根据 `VMA` 的信息，将所需的物理内存页面映射到进程的虚拟地址空间中。这通常意味着增加当前进程的地址空间大小（`sz`），并将这些新分配的内存页面标记为有效，以便后续访问。此步骤可能还涉及到页面表的更新，以确保内存映射正确。
7. 实现 `sys_munmap()`
 - i. 解析传入参数：首先，函数需要解析传入的系统调用参数，包括需要解除映射的内存起始地址和大小。这些参数将决定要解除映射的虚拟内存区域。
 - ii. 查找对应的 `VMA`：在当前进程的 `VMA` 池中查找与传入的内存地址和大小匹配的 `VMA`（虚拟内存区域）。如果找不到对应的 `VMA`，或者地址不合法，则返回错误。
 - iii. 回写数据到文件：根据找到的 `VMA` 信息，如果该 `VMA` 是与文件映射相关的（而不是匿名映射），则需要将修改过的数据回写到文件中。这一步确保了文件与内存中的数据保持一致。需要遍历映射的内存区域，将脏页（已修改的页面）写回到文件。
 - iv. 更新 `VMA` 信息：根据解除映射的大小，更新 `VMA` 中的相关信息。如果只是一部分内存被解除映射，则需要调整 `VMA` 的起始地址和大小，以反映剩余的映射区域。
 - v. 释放 `VMA` 和文件引用：如果 `munmap` 操作完全解除整个 `mmap` 的内存区域，则需要从 `VMA` 池中移除该 `VMA`，并释放与该 `VMA` 相关的资源，例如减少对映射文件的引用计数，释放物理

内存页面，更新进程的地址空间大小等。如果该文件引用计数归零，还需进一步清理相关的文件资源。

8. 在 `page fault handler` 中分配物理内存。

- i. 虚拟内存分配与页面错误：在执行 `mmap` 系统调用时，我们只分配了进程的虚拟内存区域，并未分配对应的物理内存页面。当用户第一次访问 `mmap` 分配的虚拟内存时，由于没有对应的物理内存，系统会触发页面错误（`Page Fault`）。
- ii. 在 `usertrap` 中处理页面错误：当页面错误发生时，`usertrap` 函数负责捕获并处理这个异常。我们需要在这个函数中添加页面错误的处理逻辑，以便在发生 `mmap` 相关的页面错误时为该虚拟地址分配实际的物理内存。
- iii. 因为我们实现了 `cow`，也就是 `mmap` 的内存是 `lazy allocation` 的，这导致了虚拟内存并不一定有对应的物理内存，所以需要修改一下 `vm.c` 中的 `uvmcopy` 和 `uvmunmap`。

9. 修改 `exit` 和 `fork`

实验结果

`xv6` 的 `mmaptest` 测试

实验截图：

实验心得

1. 在实现 `mmap` 系统调用时，需要确保在进程的虚拟地址空间中合理分配一个连续的虚拟内存区域，并记录该区域的信息。为了管理这些虚拟内存区域，我们引入了 `VMA` 结构体，并在 `proc` 结构体中添加了 `vma_pool` 字段。最初在设计 `VMA` 池的分配和释放机制时，如何高效地管理这些 `VMA` 条目成为一个难点。解决方案是在 `vma_pool` 中维护一个固定大小的数组，并通过标记条目的使用状态来分配和释放 `VMA`，这使得管理逻辑更加简单高效。
2. 通过本次实验，我们深入理解了操作系统中的虚拟内存管理机制，特别是 `mmap` 和 `munmap` 系统调用在实际应用中的重要性。实验中，我们不仅实现了基本的内存映射功能，还处理了页面错误、内存延迟分配、进程退出时的资源管理等复杂场景。这个过程加深了我们对虚拟内存、页面表、进程地址空间等概念的理解。

Lab9实验得分